k8s notes

1. reset k8s
    1.1. 重置原有的k8s集群
    1.2. setup calico network plugin
2. etcd操作
    2.1. etcdctl 安装
    2.2 下载etcdctl
    2.3 etcd查看member
3. 删除master node
    方案一：将其恢复为纯粹的 Worker 节点（推荐）
        1. 从 Etcd 集群中移除该成员
        2. 清理 106 上的静态 Pod
        3. 修正节点标签和角色
    方案二：正式将其设为第 4 台 Master
        1. 修正角色标签
        2. 检查污点 (Taints)
    ⚠ 核心警告：防范系统关机导致数据损坏
        接下来建议的操作：
4. k8s配置高可用和etcd备份
    4.1 集群高可用加固 (HA)
        1. 负载均衡 (Load Balancer)
        2. 修正控制面节点
    4.2 Etcd 备份策略
        1. 手动备份命令
        2. 自动化备份脚本 (Cronjob)
    4.3 容灾与数据损坏恢复 (Disaster Recovery)
        1. 停止所有 Master 的 Kubelet 和容器
        2. 移动损坏的数据
        3. 从备份恢复

4.4 针对你环境的特别建议
4.5 执行过程
4.6 把 k8s-node3-106 重新加入集群
第一步：彻底清理节点状态 (Reset)
第二步：重新执行 Join 命令
第三步：验证与后续处理
为什么之前它会运行 etcd 和 apiserver?
5. debug wordpress连接MySQL问题
5.1. debug steps:
1. 检查数据库环境变量拼写错误
2. 确认数据库 Pod 的连通性
3. 检查数据库容器的运行状态与日志
4. 手动验证数据库用户权限
5. 检查 WordPress 的 DB_HOST 配置
总结排查顺序建议：
排查结果
5.2 猜测：大概率是因为手写mydb yaml文件打错了MSYQL
第一步：验证网络连通性（最关键）
第二步：检查数据库用户权限（MySQL 8.0 特性）
第三步：重启 WordPress Pod
总结建议的排查流程
5.3 验证
5.4 其他debug方式
1. 查看容器内部的 Apache 错误日志
2. 测试数据库端口的可达性
3. 终极修复：MySQL 8.0 的“加密插件”问题
💡 为什么数据库里 show tables 是空的?
总结下一步操作：
6. 如何设置kubelet bash completion
6.1. 1. 临时生效（当前会话）
6.2. 2. 永久生效（推荐）
6.2.1. 方法 A：直接写入 .bashrc

6.2.2. 方法 B：存入系统补全目录（更规范）
6.3. 3. 注意事项与排错
6.3.1. 确保已安装 bash-completion
6.3.2. 验证是否成功
7. Pod Network Access Troubleshooting
7.1. 问题描述：
7.2. 排查过程
7.2.1. 1. CNI 网络插件未完全覆盖所有节点
7.2.2. 2. Master 节点未运行网络插件 Pod
7.2.3. 3. 安全组或防火墙（iptables/nftables）限制
7.2.4. 4. 路由不对称或反向路径过滤 (rp_filter)
7.2.5. 5. 代理/环境变量干扰
7.2.6. 建议操作步骤：
7.3. 实际排查过程
8. pod name需要小写
9. hostPort使用宿主机的端口
10. docker服务停止引起的k8s集群故障
10.1. 为什么 Docker 会好端端地停止？
10.2. 为什么一个 Master 挂了，感觉整个集群都“瘫”了？
10.3. 如何避免这些故障？（高可用设计）
10.3.1. A. 真正的多 Master 高可用
10.3.2. B. 资源预留 (Kube-reserved)
10.3.3. C. 监控与告警
10.4. 故障排查标准流程 (SOP)
11. kubectl exec vs kubectl debug
11.1. 核心区别：工作原理不同
11.2. 为什么你刚才的命令失败了？
11.3. kubectl debug 的精妙之处
11.4. 常用用法总结
11.4.1. kubectl exec：常规操作
11.4.2. kubectl debug：高级诊断
11.4.3. 范例

11.4.4. 总结建议

1. reset k8s

1.1. 重置原有的k8s集群

```shell

kubeadm reset --cri-socket=unix:///run/cri-dockerd.sock
rm -rf /etc/kubernetes
rm -rf /var/lib/kubelet
rm -rf /var/lib/etcd
rm -rf /etc/cni/net.d
rm -rf /var/lib/cni
rm -rf ~/.kube
rm -rf /root/.kube
# 删除cni虚拟网卡
ip link delete cni0 2>/dev/null
ip link delete flannel.1 2>/dev/null
ip link delete cali* 2>/dev/null
# 清空iptables k8s规则
iptables -F
iptables -t nat -F
iptables -t mangle -F
iptables -X
systemctl restart containerd
systemctl enable kubelet
systemctl restart kubelet

```

1.2. setup calico network plugin

```

  0 ✓ 14:36:01 root@k8s-master1-101,172.17.0.1:/data/scripts # docker images



                                                i Info →   U  In Use
IMAGE
 ID             DISK USAGE   CONTENT SIZE   EXTRA
alpine:latest
 5b10f432ef3d       13.1MB         3.95MB    U
ghcr.io/flannel-io/flannel-cni-plugin:v1.9.1-flannel1
 7c3377e977b4       17.3MB         5.16MB
ghcr.io/flannel-io/flannel:v0.28.4
 cc44a1a8969c        132MB         35.1MB
hello-world:latest
 f9078146db2e       25.9kB         9.49kB    U
nginx:latest
 6e23479198b9        240MB         65.8MB    U
quay.io/calico/cni:v3.31.4
 210055d96825        236MB         72.2MB    U
registry.aliyuncs.com/google_containers/coredns:v1.13.1
 6300daf5e742        104MB         22.8MB
registry.aliyuncs.com/google_containers/etcd:3.6.6-0
 60a30b5d81b2       89.5MB         23.6MB    U
registry.aliyuncs.com/google_containers/kube-apiserver:v1.35.0
 86145709a4e6        121MB         27.7MB
registry.aliyuncs.com/google_containers/kube-apiserver:v1.35.4
 5a5809a201d4        121MB         27.6MB    U
registry.aliyuncs.com/google_containers/kube-controller-manager:v1.35.0
 a7576e01c34f        102MB         23.1MB
registry.aliyuncs.com/google_containers/kube-controller-manager:v1.35.4
 13e009412895        102MB           23MB    U
registry.aliyuncs.com/google_containers/kube-proxy:v1.35.0
 9d74df50eeaf        100MB         25.8MB
registry.aliyuncs.com/google_containers/kube-proxy:v1.35.4
 9d893ec4f84b       99.9MB         25.7MB    U
registry.aliyuncs.com/google_containers/kube-scheduler:v1.35.0
 d62f59db793d       72.3MB         17.2MB
registry.aliyuncs.com/google_containers/kube-scheduler:v1.35.4
 2c4183d53f4e       72.2MB         17.1MB    U
registry.aliyuncs.com/google_containers/pause:3.10.1
 d8aed0a71d2b       1.06MB          318kB    U
  0 ✓ 14:36:08 root@k8s-master1-101,172.17.0.1:/data/scripts # docker pull quay.io/calico/kube-controllers:v3.31.4
v3.31.4: Pulling from calico/kube-controllers
20aa39c128ba: Pull complete
```

```
Digest: sha256:89d02983f8cc13661bb07b172d4e298f60f5e9c5b26e3626d8e5caa0a66b1469
Status: Downloaded newer image for quay.io/calico/kube-controllers:v3.31.4
quay.io/calico/kube-controllers:v3.31.4
  0 ✓ 14:36:45 root@k8s-master1-101,172.17.0.1:/data/scripts # docker pull quay.io/calico/node:v3.31.4
v3.31.4: Pulling from calico/node
Digest: sha256:2a3656eb74aa76a697dec178dc089462a908c66360a838513f0c00c7245c5e6f
Status: Image is up to date for quay.io/calico/node:v3.31.4
quay.io/calico/node:v3.31.4
  0 ✓ 14:36:50 root@k8s-master1-101,172.17.0.1:/data/scripts # kubectl get po -n kube-system -o wide
NAME                                       READY   STATUS    RESTARTS   AGE     IP            NODE              NOMINATED NODE   READINESS GATES
calico-kube-controllers-9dff488b-vhk9p     1/1     Running   0          40m     192.168.0.5   k8s-master-101    <none>           <none>
calico-node-89zvm                          1/1     Running   0          40m     10.0.0.101    k8s-master-101    <none>           <none>
calico-node-l45vk                          1/1     Running   0          40m     10.0.0.103    k8s-master3-103   <none>           <none>
calico-node-nftkk                          1/1     Running   0          40m     10.0.0.102    k8s-master2-102   <none>           <none>
calico-node-pth85                          1/1     Running   0          40m     10.0.0.104    k8s-node1-104     <none>           <none>
calico-node-r7w77                          1/1     Running   0          40m     10.0.0.105    k8s-node2-105     <none>           <none>
calico-node-vdg2n                          1/1     Running   0          40m     10.0.0.106    k8s-node3-106     <none>           <none>
coredns-bbdc5fdf6-wdp85                    1/1     Running   0          4h29m   192.168.0.8   k8s-master-101    <none>           <none>
coredns-bbdc5fdf6-wgjlg                    1/1     Running   0          4h29m   192.168.0.7   k8s-master-101    <none>           <none>
etcd-k8s-master-101                        1/1     Running   8 (2m23s ago)   4h29m   10.0.0.101    k8s-master-101    <none>           <none>
etcd-k8s-master2-102                       1/1     Running   1 (12m ago)   3h21m   10.0.0.102    k8s-master2-102   <none>           <none>
etcd-k8s-master3-103                       1/1     Running   0          179m    10.0.0.103    k8s-master3-103   <none>           <none>
etcd-k8s-node3-106                         1/1     Running   0          4h3m    10.0.0.106    k8s-node3-106     <none>           <none>
kube-apiserver-k8s-master-101              1/1     Running   7 (2m21s ago)   4h29m   10.0.0.101    k8s-master-101    <none>           <none>
kube-apiserver-k8s-master2-102             1/1     Running   1 (12m ago)   3h21m   10.0.0.102    k8s-master2-102   <none>           <none>
```

```
kube-apiserver-k8s-master3-103            1/1     Running   0
   179m    10.0.0.103    k8s-master3-103   <none>           <none>
kube-apiserver-k8s-node3-106              1/1     Running   0
   4h3m    10.0.0.106    k8s-node3-106     <none>           <none>
kube-controller-manager-k8s-master-101    1/1     Running   2 (2m31s ag
o)   4h29m   10.0.0.101    k8s-master-101    <none>           <none>
kube-controller-manager-k8s-master2-102   1/1     Running   1 (12m ago)
   3h21m   10.0.0.102    k8s-master2-102   <none>           <none>
kube-controller-manager-k8s-master3-103   1/1     Running   0
   179m    10.0.0.103    k8s-master3-103   <none>           <none>
kube-controller-manager-k8s-node3-106     1/1     Running   0
   4h3m    10.0.0.106    k8s-node3-106     <none>           <none>
kube-proxy-7z4hg                          1/1     Running   1 (12m ago)
   3h21m   10.0.0.102    k8s-master2-102   <none>           <none>
kube-proxy-fhjw2                          1/1     Running   1 (2m31s ag
o)   4h29m   10.0.0.101    k8s-master-101    <none>           <none>
kube-proxy-lrfdk                          1/1     Running   0
   179m    10.0.0.103    k8s-master3-103   <none>           <none>
kube-proxy-qmtsd                          1/1     Running   0
   175m    10.0.0.104    k8s-node1-104     <none>           <none>
kube-proxy-svz2l                          1/1     Running   0
   4h3m    10.0.0.106    k8s-node3-106     <none>           <none>
kube-proxy-xtqxv                          1/1     Running   0
   175m    10.0.0.105    k8s-node2-105     <none>           <none>
kube-scheduler-k8s-master-101             1/1     Running   2 (2m31s ag
o)   4h29m   10.0.0.101    k8s-master-101    <none>           <none>
kube-scheduler-k8s-master2-102            1/1     Running   1 (12m ago)
   3h21m   10.0.0.102    k8s-master2-102   <none>           <none>
kube-scheduler-k8s-master3-103            1/1     Running   0
   179m    10.0.0.103    k8s-master3-103   <none>           <none>
kube-scheduler-k8s-node3-106              1/1     Running   0
   4h3m    10.0.0.106    k8s-node3-106     <none>           <none>
  0 ✓ 14:37:01 root@k8s-master1-101,172.17.0.1:/data/scripts # kubectl ap
i-versions | grep crd
crd.projectcalico.org/v1
  0 ✓ 14:37:36 root@k8s-master1-101,172.17.0.1:/data/scripts # kubectl ap
i-resources | wc -l
90
  0 ✓ 14:37:52 root@k8s-master1-101,172.17.0.1:/data/scripts # kubectl ap
i-resources --api-group=crd.projectcalico.org
NAME                              SHORTNAMES   APIVERSION
 NAMESPACED   KIND
bgpconfigurations                              crd.projectcalico.org/v1
 false        BGPConfiguration
bgpfilters                                     crd.projectcalico.org/v1
 false        BGPFilter
bgppeers                                       crd.projectcalico.org/v1
 false        BGPPeer
```

```
blockaffinities                                   crd.projectcalico.org/v1
 false        BlockAffinity
caliconodestatuses                                crd.projectcalico.org/v1
 false        CalicoNodeStatus
clusterinformations                               crd.projectcalico.org/v1
 false        ClusterInformation
felixconfigurations                               crd.projectcalico.org/v1
 false        FelixConfiguration
globalnetworkpolicies                             crd.projectcalico.org/v1
 false        GlobalNetworkPolicy
globalnetworksets                                 crd.projectcalico.org/v1
 false        GlobalNetworkSet
hostendpoints                                     crd.projectcalico.org/v1
 false        HostEndpoint
ipamblocks                                        crd.projectcalico.org/v1
 false        IPAMBlock
ipamconfigs                                       crd.projectcalico.org/v1
 false        IPAMConfig
ipamhandles                                       crd.projectcalico.org/v1
 false        IPAMHandle
ippools                                           crd.projectcalico.org/v1
 false        IPPool
ipreservations                                    crd.projectcalico.org/v1
 false        IPReservation
kubecontrollersconfigurations                     crd.projectcalico.org/v1
 false        KubeControllersConfiguration
networkpolicies                                   crd.projectcalico.org/v1
 true         NetworkPolicy
networksets                                       crd.projectcalico.org/v1
 true         NetworkSet
stagedglobalnetworkpolicies                       crd.projectcalico.org/v1
 false        StagedGlobalNetworkPolicy
stagedkubernetesnetworkpolicies                   crd.projectcalico.org/v1
 true         StagedKubernetesNetworkPolicy
stagednetworkpolicies                             crd.projectcalico.org/v1
 true         StagedNetworkPolicy
tiers                                             crd.projectcalico.org/v1
 false        Tier
  0 ✓ 14:38:13 root@k8s-master1-101,172.17.0.1:/data/scripts # kubectl ge
t pod -n kube-system | grep calico
calico-kube-controllers-9dff488b-vhk9p    1/1     Running   0
   42m
calico-node-89zvm                         1/1     Running   0
   42m
calico-node-l45vk                         1/1     Running   0
   42m
calico-node-nftkk                         1/1     Running   0
   42m
```

```
calico-node-pth85                          1/1     Running   0
   42m
calico-node-r7w77                          1/1     Running   0
   42m
calico-node-vdg2n                          1/1     Running   0
   42m
  0 ✓ 14:38:39 root@k8s-master1-101,172.17.0.1:/data/scripts # kubectl ge
t nodes
NAME               STATUS   ROLES           AGE     VERSION
k8s-master-101     Ready    control-plane   4h31m   v1.35.4
k8s-master2-102    Ready    control-plane   3h23m   v1.35.4
k8s-master3-103    Ready    control-plane   3h      v1.35.4
k8s-node1-104      Ready    <none>          177m    v1.35.4
k8s-node2-105      Ready    <none>          177m    v1.35.4
k8s-node3-106      Ready    <none>          4h4m    v1.35.4
  0 ✓ 14:38:47 root@k8s-master1-101,172.17.0.1:/data/scripts # kubectl ge
t ippools
NAME                  AGE
default-ipv4-ippool   40m
  0 ✓ 14:38:51 root@k8s-master1-101,172.17.0.1:/data/scripts # kubectl ge
t ippools.crd.projectcalico.org default-ipv4-ippool -o yaml
apiVersion: crd.projectcalico.org/v1
kind: IPPool
metadata:
  annotations:
    projectcalico.org/metadata: '{"creationTimestamp":"2026-05-14T05:58:4
0Z"}'
  creationTimestamp: "2026-05-14T05:58:40Z"
  generation: 1
  name: default-ipv4-ippool
  resourceVersion: "22079"
  uid: dd8a3bb6-460c-4e94-afa8-98814e51d280
spec:
  allowedUses:
  - Workload
  - Tunnel
  assignmentMode: Automatic
  blockSize: 24
  cidr: 192.168.0.0/24
  ipipMode: Always
  natOutgoing: true
  nodeSelector: all()
  vxlanMode: Never
  0 ✓ 14:39:40 root@k8s-master1-101,172.17.0.1:/data/scripts #
```

2. etcd操作

2.1. etcdctl 安装

包安装 etcdctl 工具：包安装的版本可能不被新的k8集群支持（不建议）

```


127 ✗ 14:57:40 root@k8s-master1-101,172.17.0.1:/data/scripts # kubectl get po -n kube-system | grep etcd
etcd-k8s-master-101                        1/1     Running   8 (26m ago)  4h53m
etcd-k8s-master2-102                       1/1     Running   1 (36m ago)  3h46m
etcd-k8s-master3-103                       1/1     Running   0            3h23m
etcd-k8s-node3-106                         1/1     Running   0            4h27m
  0 ✓ 15:01:10 root@k8s-master1-101,172.17.0.1:/data/scripts # kubectl get pod -n kube-system etcd-k8s-master-101 -o yaml | grep image
    image: registry.aliyuncs.com/google_containers/etcd:3.6.6-0
    imagePullPolicy: IfNotPresent
    image: registry.aliyuncs.com/google_containers/etcd:3.6.6-0
    imageID: docker-pullable://registry.aliyuncs.com/google_containers/etcd@sha256:60a30b5d81b2217555e2cfb9537f655b7ba97220b99c39ee2e162a7127225890
  0 ✓ 15:01:44 root@k8s-master1-101,172.17.0.1:/data/scripts #
# etcd version checking
  0 ✓ 15:01:44 root@k8s-master1-101,172.17.0.1:/data/scripts # kubectl exec -n kube-system etcd-k8s-master-101 -- /usr/local/bin/etcd --version
etcd Version: 3.6.6
Git SHA: d2809cf
Go Version: go1.24.10
Go OS/Arch: linux/amd64
  0 ✓ 15:03:40 root@k8s-master1-101,172.17.0.1:/data/scripts #

```

2.2 下载etcdctl

```

  0 ✓ 15:03:40 root@k8s-master1-101,172.17.0.1:/data/scripts # ETCD_VER=v3.6.6
  0 ✓ 15:07:16 root@k8s-master1-101,172.17.0.1:/data/scripts # GOOGLE_URL=https://storage.googleapis.com/etcd
  0 ✓ 15:07:25 root@k8s-master1-101,172.17.0.1:/data/scripts # DOWNLOAD_URL=${GOOGLE_URL}

 23 ✗ 15:09:08 root@k8s-master1-101,172.17.0.1:/data/scripts # curl -L https://storage.googleapis.com/etcd/v3.6.6/etcd-v3.6.6-linux-amd64.tar.gz -o /tmp/etcd-${ETCD_VER}-linux-amd64.tar.gz
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 23.1M  100 23.1M    0     0  6377k      0  0:00:03  0:00:03 --:--:-- 6378k
  0 ✓ 15:09:56 root@k8s-master1-101,172.17.0.1:/data/scripts # tar xf /tmp/etcd-v3.6.6-linux-amd64.tar.gz -C /srv
  0 ✓ 15:10:42 root@k8s-master1-101,172.17.0.1:/data/scripts # ls /srv
etcd-v3.6.6-linux-amd64
  0 ✓ 15:10:46 root@k8s-master1-101,172.17.0.1:/data/scripts # ls /srv/etcd-v3.6.6-linux-amd64/
Documentation  etcd  etcdctl  etcdutl  README-etcdctl.md  README-etcdutl.md  README.md  README-etcdctl.md
  0 ✓ 15:10:52 root@k8s-master1-101,172.17.0.1:/data/scripts #


# 查看etcd版本

  0 ✓ 15:10:52 root@k8s-master1-101,172.17.0.1:/data/scripts # /srv/etcd-v3.6.6-linux-amd64/etcdctl version
etcdctl version: 3.6.6
API version: 3.6
  1 ✗ 15:12:30 root@k8s-master1-101,172.17.0.1:/data/scripts # /srv/etcd-v3.6.6-linux-amd64/etcd --version
etcd Version: 3.6.6
Git SHA: d2809cf
Go Version: go1.24.10
Go OS/Arch: linux/amd64
  0 ✓ 15:12:44 root@k8s-master1-101,172.17.0.1:/data/scripts #

```

2.3 etcd查看member

```
# 查看 etcd 成员列表
  0 ✓ 15:14:33 root@k8s-master1-101,172.17.0.1:/data/scripts # export ETCDCTL_API=3
etcdctl --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  member list
{"level":"warn","ts":"2026-05-14T15:15:19.701998+0800","caller":"flags/flag.go:94","msg":"unrecognized environment variable","environment-variable":"ETCDCTL_API=3"}
5bf970269192dca, started, k8s-master3-103, https://10.0.0.103:2380, https://10.0.0.103:2379, false
178806d05592aff5, started, k8s-master2-102, https://10.0.0.102:2380, https://10.0.0.102:2379, false
bffb85594dabfed2, started, k8s-node3-106, https://10.0.0.106:2380, https://10.0.0.106:2379, false
f2fd0c12369e0d75, started, k8s-master-101, https://10.0.0.101:2380, https://10.0.0.101:2379, false
  0 ✓ 15:15:19 root@k8s-master1-101,172.17.0.1:/data/scripts #


#移除membmer 106
  0 ✓ 15:15:19 root@k8s-master1-101,172.17.0.1:/data/scripts # etcdctl --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  member remove bffb85594dabfed2
{"level":"warn","ts":"2026-05-14T15:16:46.836227+0800","caller":"flags/flag.go:94","msg":"unrecognized environment variable","environment-variable":"ETCDCTL_API=3"}
Member bffb85594dabfed2 removed from cluster 3adf25d56faa167e
  0 ✓ 15:16:47 root@k8s-master1-101,172.17.0.1:/data/scripts # etcdctl --endpoints=https://127.0.0.1:2379   --cacert=/etc/kubernetes/pki/etcd/ca.crt   --cert=/etc/kubernetes/pki/etcd/server.crt   --key=/etc/kubernetes/pki/etcd/server.key   member list
{"level":"warn","ts":"2026-05-14T15:16:54.113277+0800","caller":"flags/flag.go:94","msg":"unrecognized environment variable","environment-variable":"ETCDCTL_API=3"}
5bf970269192dca, started, k8s-master3-103, https://10.0.0.103:2380, https://10.0.0.103:2379, false
178806d05592aff5, started, k8s-master2-102, https://10.0.0.102:2380, https://10.0.0.102:2379, false
f2fd0c12369e0d75, started, k8s-master-101, https://10.0.0.101:2380, https://10.0.0.101:2379, false
  0 ✓ 15:16:54 root@k8s-master1-101,172.17.0.1:/data/scripts #
```

```
30
```

3. 删除master node

如果需要去除一个master，需做如下操作

```
1
```

由于 `k8s-node3-106` 目前运行着 `etcd` 和 `kube-apiserver`，这说明它在集群中的身份非常尴尬：它既是 Worker 节点，又承担了 Master 的核心职能（可能是通过 `kubeadm join --control-plane` 加入的，或者手动配置了静态 Pod）。

为了集群的长期稳定，你需要二选一。以下是具体的实操步骤：

方案一：将其恢复为纯粹的 Worker 节点（推荐）

如果你的初衷是“三主三从”，那么控制面应该只在 101、102、103 上。你需要从控制面逻辑中剔除 106。

1. 从 Etcd 集群中移除该成员

这是最关键的一步。在 **k8s-master-101** 上执行：

```
# 查看 etcd 成员列表
export ETCDCTL_API=3
etcdctl --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  member list

# 找到 106 对应的 ID，并将其删除
etcdctl --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  member remove <106-NODE-ID>

```

2. 清理 106 上的静态 Pod

登录 **k8s-node3-106**，删除 Master 组件的定义文件，kubelet 会自动停止这些容器：

```
# 移除控制面清单文件
rm -f /etc/kubernetes/manifests/kube-apiserver.yaml
rm -f /etc/kubernetes/manifests/kube-controller-manager.yaml
rm -f /etc/kubernetes/manifests/kube-scheduler.yaml
rm -f /etc/kubernetes/manifests/etcd.yaml

# 移除 Master 相关的证书（可选，建议保留备份）
rm -rf /etc/kubernetes/pki/etcd/

```

3. 修正节点标签和角色

在 **k8s-master-101** 上执行，取消其控制面角色标记：

```
kubectl label node k8s-node3-106 node-role.kubernetes.io/control-plane-
kubectl label node k8s-node3-106 node-role.kubernetes.io/master-

```

方案二：正式将其设为第 4 台 Master

如果你希望保留它的 Master 职能，你需要补全它的角色标签和管理配置。**注意：Etcd 集群节点数建议为奇数（3, 5, 7），4 个节点并不能提升容错率，反而增加了网络协商开销。**

1. 修正角色标签

```
kubectl label node k8s-node3-106 node-role.kubernetes.io/control-plane=

```

2. 检查污点 (Taints)

通常 Master 不运行普通业务 Pod。如果你希望 106 既做 Master 又做 Worker（这不符合生产高可用规范，但在实验环境可行），可以保留现状。如果你希望它只做 Master：

```
kubectl taint nodes k8s-node3-106 node-role.kubernetes.io/control-plane:NoSchedule

```

⚠️ 核心警告：防范系统关机导致数据损坏

既然你担心突然关机，请务必检查以下两点：

1. **Etcd 数据持久化**：
   确认你的 `etcd` 数据目录在 `/var/lib/etcd` 。如果这个目录在根分区的虚假磁盘空间（如 tmpfs）中，关机数据即丢。
2. **强制刷盘设置**：
   Etcd 非常依赖磁盘写入速度。如果关机频繁，建议在 `etcd.yaml` 中优化参数：

- `--snapshot-count=10000`（减少快照频率，降低 IO 压力）
- 或者确保你的文件系统（如 EXT4/XFS）开启了正常的日志模式。

接下来建议的操作：

既然你已经有了三台（或四台）Master，**现在最紧迫的任务是配置负载均衡（Keepalived + HAProxy）**。否则，只要当前的 Master-101 关机，即便其他两台 Master 在线，你的 `kubectl` 命令也会因为连接不到 101 而失效。

需要我为你提供这套 **高可用 VIP 自动漂移** 的配置方案吗？

4. k8s配置高可用和etcd备份

从你提供的 `kubectl get po -n kube-system -o wide` 输出看，你的集群已经具备了初步的高可用雏形（三个 control-plane 节点），但存在一个**严重风险点**：

你的 `k8s-node3-106` 节点上竟然运行着 `etcd`、`kube-apiserver` 等核心组件（通常由 kubeadm 自动部署为静态 Pod）。在标准的三主三从架构中，**Work Node 不应承载控制面组件**。

为了确保系统在断电或损坏时能快速恢复，你需要从以下两个维度进行加固：

4.1 集群高可用加固 (HA)

目前你有三个 Master，但如果你的 `kube-config` 或 Worker 节点连接的是某一个 Master 的固定 IP（如 `10.0.0.101`），那么该节点宕机时，集群依然会“失联”。

1. 负载均衡 (Load Balancer)

你需要一个虚拟 IP (VIP) 来代理三台 Master 的 6443 端口。

- **方案 A (推荐)**：在 Master 节点上部署 `Keepalived` + `HAProxy`。
- **方案 B**：使用云厂商的 LB 或硬件 LB。

配置思路（以 Keepalived + HAProxy 为例）：

1. 在三台 Master 安装 Keepalived，配置一个 VIP（例如 `10.0.0.100`）。
2. 安装 HAProxy，将流量分发到 101/102/103 的 6443 端口。
3. **关键点**：修改所有节点上 `kubelet.conf` 和 `kube-proxy` 的配置，将 server 地址指向 VIP。

2. 修正控制面节点

从你的输出看，`k8s-node3-106` 运行了 `etcd` 和 `apiserver`。如果这是误操作加入的，建议将其角色转换回纯粹的 Worker，或者正式将其加入控制面。

- **检查副本数**：Etcd 必须是奇数（3, 5, 7）。当前你有 4 个 Etcd 实例（101, 102, 103, 106），这在 Raft 协议中是不健康的，建议移除 106 上的 Etcd 角色。

4.2 Etcd 备份策略

Etcd 是 Kubernetes 的“大脑”，防止数据损坏的唯一手段是**定期快照**。

1. 手动备份命令

在任意一台 Master 节点上执行：

```
# 需确保安装了 etcdctl 工具
export ETCDCTL_API=3
etcdctl --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  snapshot save /data/backup/etcd-snapshot-$(date +%Y%m%d).db

```

2. 自动化备份脚本 (Cronjob)

创建一个脚本 `/data/scripts/etcd-backup.sh`：

```
#!/bin/bash
BACKUP_DIR="/data/etcd-backup"
KEEP_DAYS=7
mkdir -p $BACKUP_DIR

export ETCDCTL_API=3
etcdctl --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  snapshot save $BACKUP_DIR/etcd-snapshot-$(date +%Y%m%d_%H%M%S).db

# 删除7天前的备份
find $BACKUP_DIR -name "*.db" -mtime +$KEEP_DAYS -exec rm -f {} \;

```

然后添加 crontab 任务，每小时备份一次。

- **赋予执行权限**: `chmod +x /data/scripts/etcd-backup.sh`
- **加入定时任务**: `crontab -e` 并添加一行 `0 */2 * * * /data/scripts/backup_etcd.sh` （每2小时备份一次）。

4.3 容灾与数据损坏恢复 (Disaster Recovery)

如果系统突然关机导致 Etcd 数据损坏（通常表现为 `etcd` Pod 无法启动或一直重启），按以下步骤恢复：

1. 停止所有 Master 的 Kubelet 和容器

```
systemctl stop kubelet
docker stop $(docker ps -q | grep etcd) # 或 crictl stop

```

2. 移动损坏的数据

```
mv /var/lib/etcd /var/lib/etcd.bak

```

3. 从备份恢复

在**每一台** Master 上使用 `etcdctl snapshot restore` 命令。

> **注意**：恢复时需要指定 `--initial-cluster` 等参数，确保三台 Master 重新组成一个新的集群，而不是三个独立的单点。

4.4 针对你环境的特别建议

1. **节点状态观察**：你的 `etcd-k8s-master-101` 有 8 次重启，这通常意味着磁盘 IO 性能不足或网络抖动。Etcd 对磁盘延迟非常敏感，**建议将 `/var/lib/etcd` 挂载在 SSD 上**。

2. **文件同步**：务必将生成的 `etcd` 备份文件通过 `rsync` 或 `scp` 定期传输到**集群之外**的存储（如 NAS 或另一台备份服务器）。

你需要我现在为你生成一份完整的 Keepalived + HAProxy 配置文件模版吗？

4.5 执行过程

```

  5 ✗ 15:43:11 root@k8s-master1-101,172.17.0.1:/data/scripts # # 1. 创建备份目录
mkdir -p /data/backup/

# 2. 执行备份（注意 ETCDCTL_API=3 放在最前面，且确保路径正确）
ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  snapshot save /data/backup/etcd-snapshot-$(date +%Y%m%d).db
{"level":"warn","ts":"2026-05-14T15:44:51.495520+0800","caller":"flags/flag.go:94","msg":"unrecognized environment variable","environment-variable":"ETCDCTL_API=3"}
{"level":"info","ts":"2026-05-14T15:44:51.500703+0800","caller":"snapshot/v3_snapshot.go:83","msg":"created temporary db file","path":"/data/backup/etcd-snapshot-20260514.db.part"}
{"level":"info","ts":"2026-05-14T15:44:51.507100+0800","logger":"client","caller":"v3@v3.6.6/maintenance.go:236","msg":"opened snapshot stream; downloading"}
{"level":"info","ts":"2026-05-14T15:44:51.516097+0800","caller":"snapshot/v3_snapshot.go:96","msg":"fetching snapshot","endpoint":"https://127.0.0.1:2379"}
{"level":"info","ts":"2026-05-14T15:44:51.918688+0800","logger":"client","caller":"v3@v3.6.6/maintenance.go:302","msg":"completed snapshot read; closing"}
{"level":"info","ts":"2026-05-14T15:44:51.925678+0800","caller":"snapshot/v3_snapshot.go:111","msg":"fetched snapshot","endpoint":"https://127.0.0.1:2379","size":"5.4 MB","took":"424.536853ms","etcd-version":"3.6.0"}
{"level":"info","ts":"2026-05-14T15:44:51.925797+0800","caller":"snapshot/v3_snapshot.go:121","msg":"saved","path":"/data/backup/etcd-snapshot-20260514.db"}
Snapshot saved at /data/backup/etcd-snapshot-20260514.db
Server version 3.6.0
  0 ✓ 15:44:52 root@k8s-master1-101,172.17.0.1:/data/scripts #


  # 检查状态
    0 ✓ 15:53:02 root@k8s-master1-101,172.17.0.1:/data/scripts # ETCD_API=3 etcdctl --endpoints=https://127.0.0.1:2379 --cacert=/etc/kubernetes/pki/etcd/ca.crt --cert=/etc/kubernetes/pki/etcd/server.crt --key=/etc/kubernetes/pki/etcd/server.key --write-out=table member list
{"level":"warn","ts":"2026-05-14T15:54:03.670302+0800","caller":"flags/flag.go:94","msg":"unrecognized environment variable","environment-variable":"ETCDCTL_API=3"}
+------------------+---------+------------------+------------------------+------------------------+------------+
```

```
|        ID        | STATUS  |      NAME       |       PEER ADDRS
|      CLIENT ADDRS       | IS LEARNER |
+------------------+---------+-----------------+-------------------------+
-------------------------+------------+
|  5bf970269192dca | started | k8s-master3-103 | https://10.0.0.103:2380
| https://10.0.0.103:2379 |      false |
| 178806d05592aff5 | started | k8s-master2-102 | https://10.0.0.102:2380
| https://10.0.0.102:2379 |      false |
| f2fd0c12369e0d75 | started |  k8s-master-101 | https://10.0.0.101:2380
| https://10.0.0.101:2379 |      false |
+------------------+---------+-----------------+-------------------------+
-------------------------+------------+
  0 ✓ 15:54:04 root@k8s-master1-101,172.17.0.1:/data/scripts # ETCD_API=3
etcdctl --endpoints=https://127.0.0.1:2379 --cacert=/etc/kubernetes/pki/et
cd/ca.crt --cert=/etc/kubernetes/pki/etcd/server.crt --key=/etc/kubernete
s/pki/etcd/server.key -w table endpoint status
{"level":"warn","ts":"2026-05-14T15:54:40.736309+0800","caller":"flags/fla
g.go:94","msg":"unrecognized environment variable","environment-variable":
"ETCDCTL_API=3"}
+------------------------+------------------+---------+-----------------+-
--------+--------+-----------------------+--------+-----------+-----------
-+-----------+------------+--------------------+--------+-----------------
---------+-------------------+
|        ENDPOINT        |        ID        | VERSION | STORAGE VERSION |
DB SIZE | IN USE | PERCENTAGE NOT IN USE | QUOTA  | IS LEADER | IS LEARNE
R | RAFT TERM | RAFT INDEX | RAFT APPLIED INDEX | ERRORS | DOWNGRADE TARGE
T VERSION | DOWNGRADE ENABLED |
+------------------------+------------------+---------+-----------------+-
--------+--------+-----------------------+--------+-----------+-----------
-+-----------+------------+--------------------+--------+-----------------
---------+-------------------+
| https://127.0.0.1:2379 | f2fd0c12369e0d75 |   3.6.6 |           3.6.0 |
 5.4 MB | 2.8 MB |                   48% | 2.1 GB |     false |      false
 |         7 |      50333 |              50333 |        |
         |             false |
+------------------------+------------------+---------+-----------------+-
--------+--------+-----------------------+--------+-----------+-----------
-+-----------+------------+--------------------+--------+-----------------
---------+-------------------+
  0 ✓ 15:54:41 root@k8s-master1-101,172.17.0.1:/data/scripts # ETCD_API=3
etcdctl --endpoints=https://127.0.0.1:2379 --cacert=/etc/kubernetes/pki/et
cd/ca.crt --cert=/etc/kubernetes/pki/etcd/server.crt --key=/etc/kubernete
s/pki/etcd/server.key -w table endpoint health
{"level":"warn","ts":"2026-05-14T15:55:12.484332+0800","caller":"flags/fla
g.go:94","msg":"unrecognized environment variable","environment-variable":
"ETCDCTL_API=3"}
+------------------------+--------+-------------+-------+
|        ENDPOINT        | HEALTH |    TOOK     | ERROR |
```

```
+------------------------+--------+-------------+-------+
| https://127.0.0.1:2379 |   true | 30.731598ms |       |
+------------------------+--------+-------------+-------+
  0 ✓ 15:55:12 root@k8s-master1-101,172.17.0.1:/data/scripts #
```

4.6 把 `k8s-node3-106` 重新加入集群

看来你想把 `k8s-node3-106` 重新加入集群（或者是想修正它的角色），但遇到了 `kubeadm join` 的典型报错。

报错的原因非常直接：**这台机器上已经残留了旧的 Kubernetes 配置和证书，且 kubelet 已经在运行中占用了端口。** Kubernetes 为了防止意外覆盖，在检测到 `/etc/kubernetes/` 下有文件时会强制退出。

要解决这个问题并将其角色转为纯粹的 Worker 或重新加入，请按以下步骤操作：

第一步：彻底清理节点状态 (Reset)

在 **k8s-node3-106** 上执行以下命令，将节点重置为“干净”状态：

```bash
# 1. 停止相关服务
systemctl stop kubelet

# 2. 使用 kubeadm 重置节点（会自动清理大部分配置）
kubeadm reset --cri-socket=unix:///run/cri-dockerd.sock

# 3. 手动清理残留文件（非常重要，kubeadm reset 不会删证书目录）
rm -rf /etc/kubernetes/
rm -rf /var/lib/kubelet/
rm -rf /var/lib/etcd/
rm -rf /etc/cni/net.d/

# 4. 清理 IPVS/IPTABLES 规则（可选，建议做）
iptables -F && iptables -t nat -F && iptables -t mangle -F && iptables -X
ipvsadm --clear

```

第二步：重新执行 Join 命令

现在这台机器干净了，你可以再次执行你的 `join` 命令。如果你想让它作为一个 **Worker 节点**（不运行 Master 组件），直接执行你之前的命令即可：

```
kubeadm join kubeapi.zhong.org:6443 --token tv2yhk.9k9usqc5mfywy6cb \
    --discovery-token-ca-cert-hash sha256:1d8b7d7f82c64fd0e93d9624d195e8cce22c4a598623aeeaf0fb5c360adbeb7 \
    --cri-socket=unix:///run/cri-dockerd.sock

```

> **注意：** 如果你的 Token 过期了（有效期通常 24h），请在 Master 节点执行 `kubeadm token create --print-join-command` 获取最新的命令。

第三步：验证与后续处理

1. **验证状态：**
   在 Master 节点上执行 `kubectl get nodes`，你会看到 `k8s-node3-106` 已经加入，且 `ROLES` 列应该是 `<none>`（代表 Worker）。
2. **清理旧记录：**
   如果你之前在 Master 节点上看到 106 已经是 Ready 状态，但现在重新 Join 了，可能会出现重复的节点名称。建议 Join 之前先在 Master 上执行：
   `kubectl delete node k8s-node3-106`

为什么之前它会运行 etcd 和 apiserver?

从你之前的截图看，它运行了控制面组件。这通常是因为：

- 你在该节点执行过 `kubeadm init`。
- 或者你在 Join 时带了 `--control-plane` 参数。

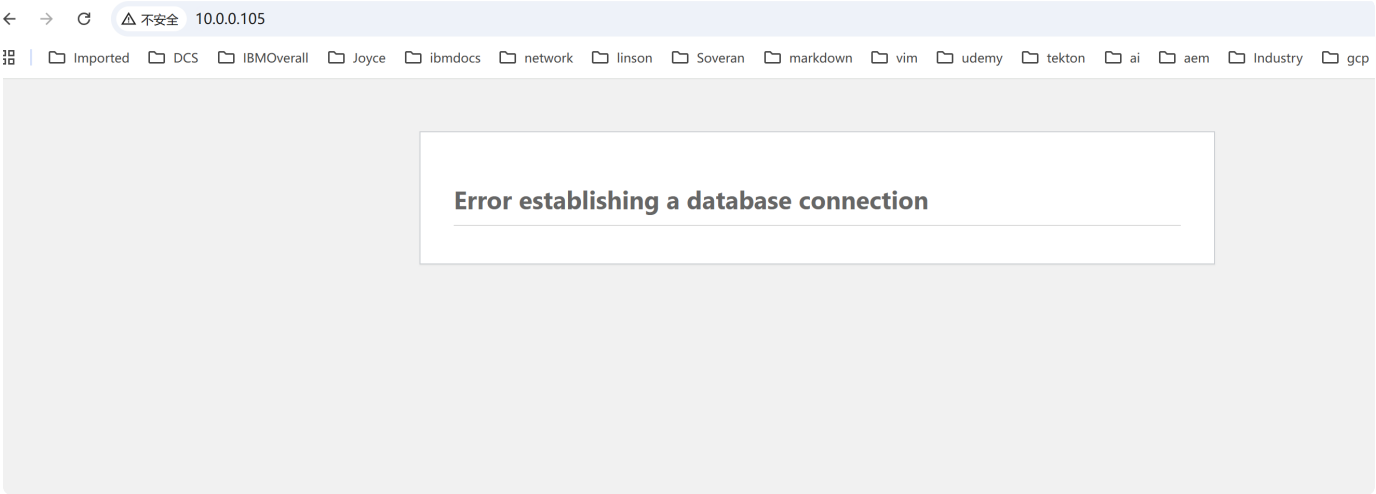
如果你现在的目标是“三主三从”：

- **101, 102, 103**：作为 Master。
- **104, 105, 106**：作为 Worker（按照上述 Reset 后重新 Join 的方法操作）。

你需要我帮你检查一下当前控制面（101–103）的健康状态，以确保重新加入 106 不会影响集群吗?

5. debug wordpress连接MySQL问题

首先设置好wordpress和mysql后，我在宿主机浏览器输入http://10.0.0.105看到如下错误



```

  1 ✗ 17:10:09 root@k8s-master2-102,172.17.0.1:~ # kubectl get po mydb -o wide -n wordpress
NAME   READY   STATUS    RESTARTS   AGE     IP             NODE            NOMINATED NODE   READINESS GATES
mydb   1/1     Running   0          2m33s   192.168.0.11   k8s-node3-106   <none>           <none>
  0 ✓ 17:10:24 root@k8s-master2-102,172.17.0.1:~ # vim pod-wordpress.yaml
  0 ✓ 17:14:36 root@k8s-master2-102,172.17.0.1:~ # kubectl apply -f pod-wordpress.yaml
pod/wordpress created
  0 ✓ 17:14:47 root@k8s-master2-102,172.17.0.1:~ # kubectl get po -n wordpress
NAME        READY   STATUS    RESTARTS   AGE
mydb        1/1     Running   0          7m3s
wordpress   1/1     Running   0          7s
  0 ✓ 17:14:54 root@k8s-master2-102,172.17.0.1:~ # kubectl get wordpress -o wide
error: the server doesn't have a resource type "wordpress"
  1 ✗ 17:15:03 root@k8s-master2-102,172.17.0.1:~ # kubectl get wordpress -o wide -n wordpress
error: the server doesn't have a resource type "wordpress"
  1 ✗ 17:15:08 root@k8s-master2-102,172.17.0.1:~ # kubectl get pod wordpress -o wide -n wordpress
NAME        READY   STATUS    RESTARTS   AGE   IP           NODE            NOMINATED NODE   READINESS GATES
wordpress   1/1     Running   0          34s   10.0.0.105   k8s-node2-105   <none>           <none>
  0 ✓ 17:15:21 root@k8s-master2-102,172.17.0.1:~ # kubectl logs wordpress
error: error from server (NotFound): pods "wordpress" not found in namespace "default"
  1 ✗ 17:16:23 root@k8s-master2-102,172.17.0.1:~ # kubectl logs wordpress -n wordpress
WordPress not found in /var/www/html - copying now...
Complete! WordPress has been successfully copied to /var/www/html
No 'wp-config.php' found in /var/www/html, but 'WORDPRESS_...' variables supplied; copying 'wp-config-docker.php' (WORDPRESS_DB_HOST WORDPRESS_DB_NAME WORDPRESS_DB_PASSWORD WORDPRESS_DB_USER)
AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 10.0.0.105. Set the 'ServerName' directive globally to suppress this message
AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 10.0.0.105. Set the 'ServerName' directive globally to suppress this message
[Thu May 14 17:14:51.762060 2026] [mpm_prefork:notice] [pid 1] AH00163: Apache/2.4.59 (Debian) PHP/8.2.20 configured -- resuming normal operations

```

```
[Thu May 14 17:14:51.762172 2026] [core:notice] [pid 1] AH00094: Command line: 'apache2 -D FOREGROUND'
10.0.0.1 - - [14/May/2026:17:16:02 +0800] "GET / HTTP/1.1" 500 2719 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
10.0.0.1 - - [14/May/2026:17:16:02 +0800] "GET /favicon.ico HTTP/1.1" 500 2719 "http://10.0.0.105/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
  0 ✓ 17:16:27 root@k8s-master2-102,172.17.0.1:~ # kubectl describe pod wordpress -n wordpress
Name:             wordpress
Namespace:        wordpress
Priority:         0
Service Account:  default
Node:             k8s-node2-105/10.0.0.105
Start Time:       Thu, 14 May 2026 17:14:48 +0800
Labels:           <none>
Annotations:      <none>
Status:           Running
IP:               10.0.0.105
IPs:
  IP:  10.0.0.105
Containers:
  wordpress:
    Container ID:   docker://7fe54700de1e2e6a53a89c4ae0424fc0bca55bd8f085bb9753ad4cebe49f5d48
    Image:          registry.cn-beijing.aliyuncs.com/wangxiaochun/wordpress:php8.2-apache
    Image ID:       docker-pullable://registry.cn-beijing.aliyuncs.com/wangxiaochun/wordpress@sha256:adced536588e066a3508dba8d3daf2715fb014d8d038e41f114a84ee3ddbbef7
    Port:           <none>
    Host Port:      <none>
    State:          Running
      Started:      Thu, 14 May 2026 17:14:49 +0800
    Ready:          True
    Restart Count:  0
    Environment:
      TZ:                     Asia/Shanghai
      WORDPRESS_DB_HOST:      192.168.0.11
      WORDPRESS_DB_NAME:      wordpress
      WORDPRESS_DB_USER:      wpuser
      WORDPRESS_DB_PASSWORD:  123456
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-spqz5 (ro)
Conditions:
  Type                        Status
```

```
  PodReadyToStartContainers   True
  Initialized                 True
  Ready                       True
  ContainersReady             True
  PodScheduled                True
Volumes:
  kube-api-access-spqz5:
    Type:                    Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds:  3607
    ConfigMapName:           kube-root-ca.crt
    Optional:                false
    DownwardAPI:             true
QoS Class:                   BestEffort
Node-Selectors:              <none>
Tolerations:                 node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                             node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type    Reason     Age   From               Message
  ----    ------     ----  ----               -------
  Normal  Scheduled  2m5s  default-scheduler  Successfully assigned wordpress/wordpress to k8s-node2-105
  Normal  Pulled     2m3s  kubelet            spec.containers{wordpress}: Container image "registry.cn-beijing.aliyuncs.com/wangxiaochun/wordpress:php8.2-apache" already present on machine and can be accessed by the pod
  Normal  Created    2m3s  kubelet            spec.containers{wordpress}: Container created
  Normal  Started    2m3s  kubelet            spec.containers{wordpress}: Container started
  0 ✓ 17:16:52 root@k8s-master2-102,172.17.0.1:~ # kubectl get pod mydb -o wide -n wordpress
NAME   READY   STATUS    RESTARTS   AGE     IP             NODE          NOMINATED NODE   READINESS GATES
mydb   1/1     Running   0          9m19s   192.168.0.11   k8s-node3-106   <none>           <none>
  0 ✓ 17:17:11 root@k8s-master2-102,172.17.0.1:~ # cat pod-wordpress.yaml
apiVersion: v1
kind: Pod
metadata:
  name: wordpress
  namespace: wordpress
spec:
  hostNetwork: true
  containers:
  - name: wordpress
```

```
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/wordpress:php8.2-apache
    env:
    - name: TZ
      value: Asia/Shanghai
    - name: WORDPRESS_DB_HOST
      value: 192.168.0.11
    - name: WORDPRESS_DB_NAME
      value: wordpress
    - name: WORDPRESS_DB_USER
      value: wpuser
    - name: WORDPRESS_DB_PASSWORD
      value: "123456"
  0 ✓ 17:17:47 root@k8s-master2-102,172.17.0.1:~ # cat pod-mysql.yaml
apiVersion: v1
kind: Pod
metadata:
  name: mydb
  namespace: wordpress
spec:
  containers:
  - name: mysql
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/mysql:8.0.29-oracle
    env:
    - name: MYSQL_ROOT_PASSWORD
      value: "654321"
    - name: MSYQL_DATABASE
      value: wordpress
    - name: MYSQL_USER
      value: wpuser
    - name: MYSQL_PASSWORD
      value: "123456"

  0 ✓ 17:17:52 root@k8s-master2-102,172.17.0.1:~ #
```

从你提供的日志和 YAML 配置来看，由于你的 `wordpress` 容器返回了 **HTTP 500** 错误，并且你提到无法连接数据库，这通常是由于网络连通性、数据库配置或权限问题导致的。

以下是针对你当前环境（Kubernetes 静态 Pod 部署）的详细排查步骤：

5.1. debug steps:

1. 检查数据库环境变量拼写错误

在你的 `pod-mysql.yaml` 中存在一个非常隐蔽的**拼写错误**：

- **错误配置：** `- name: MSYQL_DATABASE`
- **正确配置：** `- name: MYSQL_DATABASE`

因为拼写错误，MySQL 镜像可能没有在启动时自动创建名为 `wordpress` 的数据库。

解决方法： 修改 `pod-mysql.yaml`，纠正拼写，然后删除并重新创建 `mydb` Pod。

---

2. 确认数据库 Pod 的连通性

你的 WordPress Pod 使用的是 `hostNetwork: true`（IP: `10.0.0.105`），而数据库 Pod 使用的是容器网络（IP: `192.168.0.11`）。

请进入 WordPress 容器内部测试是否能连通数据库端口：

Bash

```
kubectl exec -it wordpress -n wordpress -- bash
# 在容器内执行（假设数据库使用默认 3306 端口）
curl -v 192.168.0.11:3306
```

- 如果显示 `Connected` 或有 MySQL 的握手包输出，说明网络层是通的。
- 如果显示 `Connection refused` 或超时，说明跨节点/跨网络的通信存在问题。

---

3. 检查数据库容器的运行状态与日志

查看数据库是否报错，或者是否有权限拒绝的记录：

Bash

```
kubectl logs mydb -n wordpress
```

重点观察：

- 是否有 `Ready for connections` 字样。
- 是否有关于 `wpuser` 登录失败的 `Access denied` 错误。

4. 手动验证数据库用户权限

即使 Pod 运行正常，`wpuser` 可能没有被正确授予远程访问权限（尤其是 MySQL 8.0 默认验证插件较严）。你可以进入数据库容器手动确认：

Bash

```
kubectl exec -it mydb -n wordpress -- mysql -u root -p654321
```

进入 MySQL 后执行：

SQL

```
-- 检查数据库是否存在
SHOW DATABASES;

-- 检查用户是否存在以及允许登录的主机（是否为 '%'）
SELECT user, host FROM mysql.user WHERE user='wpuser';

-- 确保权限正确
GRANT ALL PRIVILEGES ON wordpress.* TO 'wpuser'@'%';
FLUSH PRIVILEGES;
```

5. 检查 WordPress 的 DB_HOST 配置

在你的 `pod-wordpress.yaml` 中：

- `WORDPRESS_DB_HOST` 设置为 `192.168.0.11`。
- 由于你没有使用 Kubernetes Service 发现机制，如果 `mydb` Pod 重建，IP 可能会变。

建议方案：

在生产或正式练习中，建议为 `mydb` 创建一个 **Service**，然后 WordPress 通过 Service Name 访问。但就目前而言，请确保 `192.168.0.11` 确实是 `kubectl get po -o wide` 看到的最新 IP。

总结排查顺序建议：

1. **首选：** 修复 `pod-mysql.yaml` 中的 `MSYQL_DATABASE` 拼写错误。
2. **其次：** 重启 `mydb` Pod。
3. **再次：** 进入 `wordpress` 容器 `ping` 或 `curl` 数据库 IP，排除 `hostNetwork` 带来的网络隔离问题。
4. **最后：** 确认数据库内 `wpuser` 的权限。

你可以先尝试修改拼写错误，这大概率是导致 "Error establishing a database connection" 的直接原因，因为 WordPress 找不到它预想的数据库。

排查结果

```

wordpress   1/1     Running   0          34s   10.0.0.105   k8s-node2-105   <none>           <none>
  0 ✓ 17:15:21 root@k8s-master2-102,172.17.0.1:~ # kubectl logs wordpress
error: error from server (NotFound): pods "wordpress" not found in namespace "default"
  1 ✗ 17:16:23 root@k8s-master2-102,172.17.0.1:~ # kubectl logs wordpress -n wordpress
WordPress not found in /var/www/html - copying now...
Complete! WordPress has been successfully copied to /var/www/html
No 'wp-config.php' found in /var/www/html, but 'WORDPRESS_...' variables supplied; copying 'wp-config-docker.php' (WORDPRESS_DB_HOST WORDPRESS_DB_NAME WORDPRESS_DB_PASSWORD WORDPRESS_DB_USER)
AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 10.0.0.105. Set the 'ServerName' directive globally to suppress this message
AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 10.0.0.105. Set the 'ServerName' directive globally to suppress this message
[Thu May 14 17:14:51.762060 2026] [mpm_prefork:notice] [pid 1] AH00163: Apache/2.4.59 (Debian) PHP/8.2.20 configured -- resuming normal operations
[Thu May 14 17:14:51.762172 2026] [core:notice] [pid 1] AH00094: Command line: 'apache2 -D FOREGROUND'
10.0.0.1 - - [14/May/2026:17:16:02 +0800] "GET / HTTP/1.1" 500 2719 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
10.0.0.1 - - [14/May/2026:17:16:02 +0800] "GET /favicon.ico HTTP/1.1" 500 2719 "http://10.0.0.105/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
  0 ✓ 17:16:27 root@k8s-master2-102,172.17.0.1:~ # kubectl describe pod wordpress -n wordpress
Name:             wordpress
Namespace:        wordpress
Priority:         0
Service Account:  default
Node:             k8s-node2-105/10.0.0.105
Start Time:       Thu, 14 May 2026 17:14:48 +0800
Labels:           <none>
Annotations:      <none>
Status:           Running
IP:               10.0.0.105
IPs:
  IP:  10.0.0.105
Containers:
  wordpress:
    Container ID:   docker://7fe54700de1e2e6a53a89c4ae0424fc0bca55bd8f085bb9753ad4cebe49f5d48
```

```
    Image:          registry.cn-beijing.aliyuncs.com/wangxiaochun/wordpress:php8.2-apache
    Image ID:       docker-pullable://registry.cn-beijing.aliyuncs.com/wangxiaochun/wordpress@sha256:adced536588e066a3508dba8d3daf2715fb014d8d038e41f114a84ee3ddbbef7
    Port:           <none>
    Host Port:      <none>
    State:          Running
      Started:      Thu, 14 May 2026 17:14:49 +0800
    Ready:          True
    Restart Count:  0
    Environment:
      TZ:                     Asia/Shanghai
      WORDPRESS_DB_HOST:      192.168.0.11
      WORDPRESS_DB_NAME:      wordpress
      WORDPRESS_DB_USER:      wpuser
      WORDPRESS_DB_PASSWORD:  123456
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-spqz5 (ro)
Conditions:
  Type                        Status
  PodReadyToStartContainers   True
  Initialized                 True
  Ready                       True
  ContainersReady             True
  PodScheduled                True
Volumes:
  kube-api-access-spqz5:
    Type:                    Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds:  3607
    ConfigMapName:           kube-root-ca.crt
    Optional:                false
    DownwardAPI:             true
QoS Class:                   BestEffort
Node-Selectors:              <none>
Tolerations:                 node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                             node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type    Reason     Age   From               Message
  ----    ------     ----  ----               -------
  Normal  Scheduled  2m5s  default-scheduler  Successfully assigned wordpress/wordpress to k8s-node2-105
  Normal  Pulled     2m3s  kubelet            spec.containers{wordpress}: Container image "registry.cn-beijing.aliyuncs.com/wangxiaochun/wordpr
```

```
ess:php8.2-apache" already present on machine and can be accessed by the pod
  Normal  Created    2m3s  kubelet            spec.containers{wordpress}: Container created
  Normal  Started    2m3s  kubelet            spec.containers{wordpress}: Container started
  0 ✓ 17:16:52 root@k8s-master2-102,172.17.0.1:~ # kubectl get pod mydb -o wide -n wordpress
NAME   READY   STATUS    RESTARTS   AGE     IP             NODE              NOMINATED NODE   READINESS GATES
mydb   1/1     Running   0          9m19s   192.168.0.11   k8s-node3-106   <none>           <none>
  0 ✓ 17:17:11 root@k8s-master2-102,172.17.0.1:~ # cat pod-wordpress.yaml
apiVersion: v1
kind: Pod
metadata:
  name: wordpress
  namespace: wordpress
spec:
  hostNetwork: true
  containers:
  - name: wordpress
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/wordpress:php8.2-apache
    env:
    - name: TZ
      value: Asia/Shanghai
    - name: WORDPRESS_DB_HOST
      value: 192.168.0.11
    - name: WORDPRESS_DB_NAME
      value: wordpress
    - name: WORDPRESS_DB_USER
      value: wpuser
    - name: WORDPRESS_DB_PASSWORD
      value: "123456"
  0 ✓ 17:17:47 root@k8s-master2-102,172.17.0.1:~ # cat pod-mysql.yaml
apiVersion: v1
kind: Pod
metadata:
  name: mydb
  namespace: wordpress
spec:
  containers:
  - name: mysql
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/mysql:8.0.29-oracle
    env:
    - name: MYSQL_ROOT_PASSWORD
```

```
      value: "654321"
    - name: MSYQL_DATABASE
      value: wordpress
    - name: MYSQL_USER
      value: wpuser
    - name: MYSQL_PASSWORD
      value: "123456"

  0 ✓ 17:17:52 root@k8s-master2-102,172.17.0.1:~ # kubectl exec -it wordpress -n wordpress -- bash
root@k8s-node2-105:/var/www/html# curl -v 192.168.0.11:3306
*   Trying 192.168.0.11:3306...
* Connected to 192.168.0.11 (192.168.0.11) port 3306 (#0)
> GET / HTTP/1.1
> Host: 192.168.0.11:3306
> User-Agent: curl/7.88.1
> Accept: */*
>
* Received HTTP/0.9 when not allowed
* Closing connection 0
curl: (1) Received HTTP/0.9 when not allowed
root@k8s-node2-105:/var/www/html# exit
exit
command terminated with exit code 1
  1 ✗ 17:22:42 root@k8s-master2-102,172.17.0.1:~ # kubectl exec -it mydb -n wordpress -- mysql -uroot -p654321
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 11
Server version: 8.0.29 MySQL Community Server - GPL

Copyright (c) 2000, 2022, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> shwo databases;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'shwo databases' at line 1
mysql> show databases;
+--------------------+
| Database           |
```

```
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
4 rows in set (0.00 sec)

mysql> select user,host from mysql.user where user='wpuser';
+--------+------+
| user   | host |
+--------+------+
| wpuser | %    |
+--------+------+
1 row in set (0.01 sec)

mysql>
```

5.2 猜测：大概率是因为手写mydb yaml文件打错了MSYQL

更改pod-mysql.yaml文件然后删除原来的mydb pod，重建这个pod

```


  0 ✓ 17:29:57 root@k8s-master2-102,172.17.0.1:~ # vim pod-mysql.yaml
  0 ✓ 17:30:35 root@k8s-master2-102,172.17.0.1:~ # kubectl apply -f pod-mysql.yaml
The Pod "mydb" is invalid: spec: Forbidden: pod updates may not change fields other than `spec.containers[*].image`,`spec.initContainers[*].image`,`spec.activeDeadlineSeconds`,`spec.tolerations` (only additions to existing tolerations),`spec.terminationGracePeriodSeconds` (allow it to be set to 1 if it was previously negative)
@@ -105,7 +105,7 @@
      "ValueFrom": null
     },
     {
-     "Name": "MSYQL_DATABASE",
+     "Name": "MYSQL_DATABASE",
      "Value": "wordpress",
      "ValueFrom": null
     },

  1 ✗ 17:30:42 root@k8s-master2-102,172.17.0.1:~ # kubectl delete pod mydb -n wordpress
pod "mydb" deleted from wordpress namespace
  0 ✓ 17:31:09 root@k8s-master2-102,172.17.0.1:~ # kubectl apply -f pod-mysql.yaml
pod/mydb created
  0 ✓ 17:31:11 root@k8s-master2-102,172.17.0.1:~ # kubectl get po -n wordpress
NAME        READY   STATUS    RESTARTS   AGE
mydb        1/1     Running   0          7s
wordpress   1/1     Running   0          16m
  0 ✓ 17:31:18 root@k8s-master2-102,172.17.0.1:~ # kubectl logs wordpress -n wordpress
WordPress not found in /var/www/html - copying now...
Complete! WordPress has been successfully copied to /var/www/html
No 'wp-config.php' found in /var/www/html, but 'WORDPRESS_...' variables supplied; copying 'wp-config-docker.php' (WORDPRESS_DB_HOST WORDPRESS_DB_NAME WORDPRESS_DB_PASSWORD WORDPRESS_DB_USER)
AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 10.0.0.105. Set the 'ServerName' directive globally to suppress this message
AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 10.0.0.105. Set the 'ServerName' directive globally to suppress this message
[Thu May 14 17:14:51.762060 2026] [mpm_prefork:notice] [pid 1] AH00163: Apache/2.4.59 (Debian) PHP/8.2.20 configured -- resuming normal operations

```

```
[Thu May 14 17:14:51.762172 2026] [core:notice] [pid 1] AH00094: Command line: 'apache2 -D FOREGROUND'
10.0.0.1 - - [14/May/2026:17:16:02 +0800] "GET / HTTP/1.1" 500 2719 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
10.0.0.1 - - [14/May/2026:17:16:02 +0800] "GET /favicon.ico HTTP/1.1" 500 2719 "http://10.0.0.105/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
  0 ✓ 17:31:33 root@k8s-master2-102,172.17.0.1:~ # kubectl exec -it mydb -- /bin/bash
Error from server (NotFound): pods "mydb" not found
  1 ✗ 17:31:46 root@k8s-master2-102,172.17.0.1:~ # kubectl exec -it mydb -n wordpress -- /bin/bash
bash-4.4# mysql -uroot -p654321
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.0.29 MySQL Community Server - GPL

Copyright (c) 2000, 2022, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sys                |
| wordpress          |
+--------------------+
5 rows in set (0.00 sec)

mysql> show tables from wordpress;
Empty set (0.00 sec)

mysql>
```

重启pod后，继续访问还是出错

你已经成功修复了 `MYSQL_DATABASE` 的拼写错误，并且在数据库中看到了 `wordpress` 库，这说明数据库初始化已经没问题了。

目前 WordPress 仍然报错（500 错误且数据表为空），是因为 **WordPress Pod 可能在数据库准备好之前就已经启动失败并“卡死”了连接状态**，或者网络连通性依然存在障碍。

请按照以下三个步骤深度排查：

第一步：验证网络连通性（最关键）

由于你的 WordPress 在 `k8s-node2-105`，而数据库在 `k8s-node3-106`，且 WordPress 使用了 `hostNetwork: true`。我们需要确认 WordPress 容器是否能真正访问到数据库 IP。

执行以下命令进入 WordPress 容器：

```
kubectl exec -it wordpress -n wordpress -- bash

```

在容器内尝试连接数据库端口：

```
# 如果没有安装 telnet 或 nc，可以用 bash 伪设备测试
timeout 1 bash -c 'cat < /dev/null > /dev/tcp/192.168.0.11/3306' && echo "Port is Open" || echo "Connection Failed"

```

- **如果显示 Connection Failed：** 说明是网络插件（如 Calico/Flannel）或防火墙限制了跨节点的容器通信。
- **如果显示 Port is Open：** 说明网络没问题，请看下一步。

第二步：检查数据库用户权限（MySQL 8.0 特性）

MySQL 8.0 对远程连接有严格限制。虽然你在本地能登录，但需要确认 `wpuser` 是否允许从 `10.0.0.105`（WordPress 宿主机 IP）连接。

再次进入 `mydb` 执行：

```sql
-- 登录 mysql 后执行
SELECT user, host, plugin FROM mysql.user WHERE user='wpuser';

```

如果 `host` 是 `localhost` 而不是 `%`，WordPress 将无法连接。
执行以下命令修复权限和验证插件问题（WordPress 有时不支持 MySQL 8 默认的 `caching_sha2_password`）：

```sql
-- 修改为允许任意主机登录，并使用传统兼容的密码插件
ALTER USER 'wpuser'@'%' IDENTIFIED WITH mysql_native_password BY '123456';
GRANT ALL PRIVILEGES ON wordpress.* TO 'wpuser'@'%';
FLUSH PRIVILEGES;

```

第三步：重启 WordPress Pod

这是一个“玄学”但有效的步骤。
WordPress 容器在初次启动时，如果连接数据库失败，有时会生成一个不完整的 `wp-config.php`，或者 PHP-FPM 的进程没有正确重试。

既然你现在数据库已经修好了，**建议删除并重建 WordPress Pod**，让它重新执行初始化脚本：

```bash
kubectl delete -f pod-wordpress.yaml
kubectl apply -f pod-wordpress.yaml

```

总结建议的排查流程

1. **检查 IP 是否漂移：** 确认 `kubectl get po -o wide` 看到的 `mydb` IP 是否依然是 `192.168.0.11`。如果 Pod 重建了，IP 可能会变，你需要更新 `pod-wordpress.yaml` 里的 `WORDPRESS_DB_HOST`。
2. **查看详细 PHP 错误：** 如果还是 500 错误，进入 WordPress 容器查看 Apache 错误日志：
   `kubectl exec -it wordpress -n wordpress -- tail -f /var/log/apache2/error.log`
   这会告诉你具体的 PHP 报错信息（比如：Access denied for user... 或 Connection refused）。

你的 `wordpress` 库目前是 `Empty set` 是正常的，因为只有当 WordPress 成功连接上数据库后，它才会开始创建 `wp_users`，`wp_posts` 等表。只要连接通了，这些表会自动出现。

现在，你进入 WordPress 容器尝试 `ping 192.168.0.11` 看看通不通?

5.3 验证

```
  0 ✓ 17:34:09 root@k8s-master2-102,172.17.0.1:~ # kubectl exec -it wordpress -n wordpress -- bash
root@k8s-node2-105:/var/www/html# timeout 1 bash -c 'cat < /dev/null > /dev/tcp/192.168.0.11/3306'
root@k8s-node2-105:/var/www/html# timeout 1 bash -c 'cat < /dev/null > /dev/tcp/192.168.0.11/3306' && echo "Port is open"
root@k8s-node2-105:/var/www/html# # 如果没有安装 telnet 或 nc，可以用 bash 伪设备测试
timeout 1 bash -c 'cat < /dev/null > /dev/tcp/192.168.0.11/3306' && echo "Port is Open" || echo "Connection Failed"
Connection Failed
root@k8s-node2-105:/var/www/html#


  0 ✓ 17:38:18 root@k8s-master2-102,172.17.0.1:~ # kubectl exec -it mydb -n wordpress -- /bin/bash
bash-4.4# mysql -uroot -p654321
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 9
Server version: 8.0.29 MySQL Community Server - GPL

Copyright (c) 2000, 2022, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> select user,host,plugin from mysql.user where user="wpuser';
    "> ^C
mysql> select user,host,plugin from mysql.user where user='wpuser';
+--------+------+-----------------------+
| user   | host | plugin                |
+--------+------+-----------------------+
| wpuser | %    | caching_sha2_password |
+--------+------+-----------------------+
1 row in set (0.00 sec)

mysql>
```

尝试删除wordpress重建这个pod， 删除以后还是没用，结果发现是数据库的ip地址变了

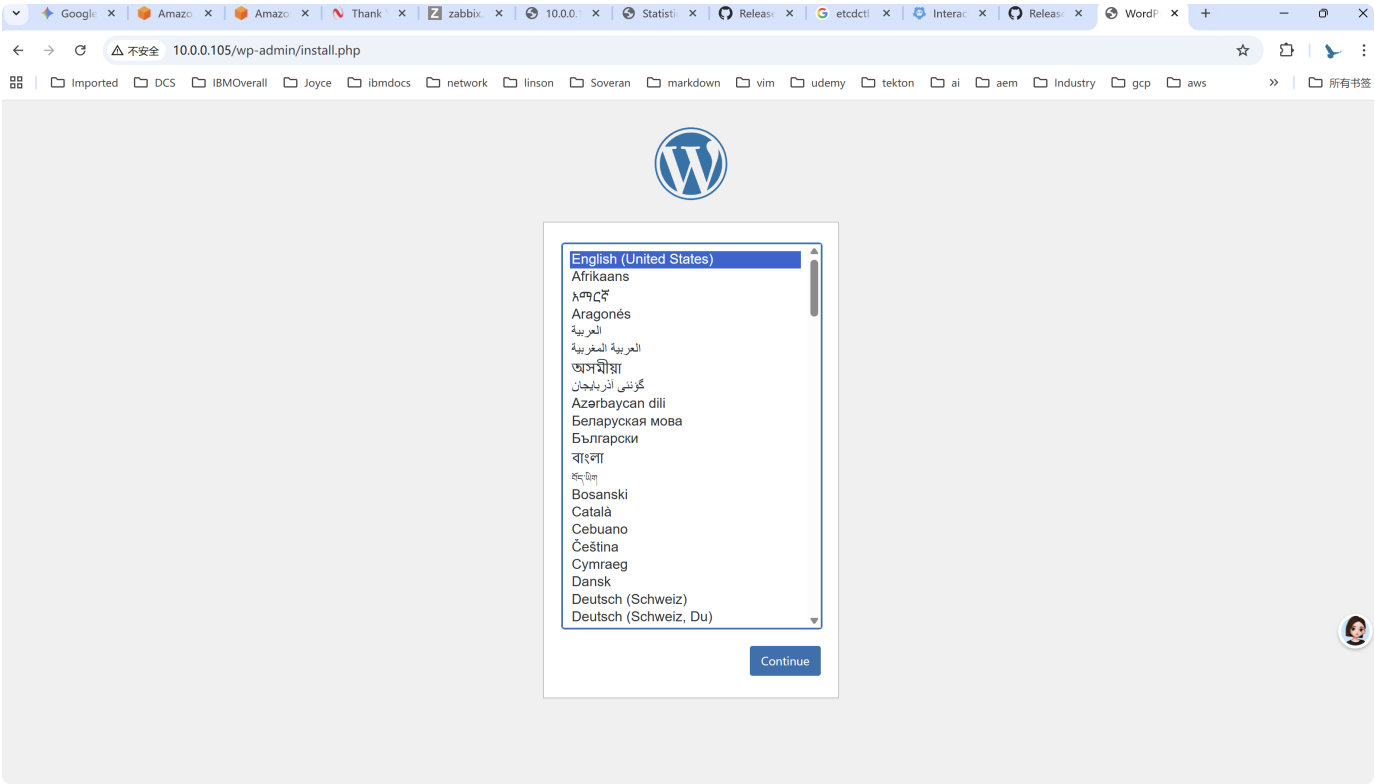
```

  0 ✓ 17:44:22 root@k8s-master2-102,172.17.0.1:~ # kubectl get po -o wide -n wordpress
NAME        READY   STATUS    RESTARTS   AGE    IP             NODE          NOMINATED NODE   READINESS GATES
mydb        1/1     Running   0          14m    192.168.0.12   k8s-node3-106   <none>           <none>
wordpress   1/1     Running   0          3m9s   10.0.0.105     k8s-node2-105   <none>           <none>
  0 ✓ 17:45:13 root@k8s-master2-102,172.17.0.1:~ # vim pod-wordpress.yaml
  0 ✓ 17:45:48 root@k8s-master2-102,172.17.0.1:~ # kubectl apply -f pod-wordpress.yaml
The Pod "wordpress" is invalid: spec: Forbidden: pod updates may not change fields other than `spec.containers[*].image`,`spec.initContainers[*].image`,`spec.activeDeadlineSeconds`,`spec.tolerations` (only additions to existing tolerations),`spec.terminationGracePeriodSeconds` (allow it to be set to 1 if it was previously negative)
@@ -106,7 +106,7 @@
     },
     {
      "Name": "WORDPRESS_DB_HOST",
-     "Value": "192.168.0.11",
+     "Value": "192.168.0.12",
      "ValueFrom": null
     },
     {

  1 ✗ 17:45:53 root@k8s-master2-102,172.17.0.1:~ # kubectl delete pod -n wordpress wordpress
pod "wordpress" deleted from wordpress namespace
  0 ✓ 17:46:16 root@k8s-master2-102,172.17.0.1:~ # kubectl apply -f pod-wordpress.yaml
pod/wordpress created
  0 ✓ 17:46:19 root@k8s-master2-102,172.17.0.1:~ # kubectl get po -n wordpress
NAME        READY   STATUS    RESTARTS   AGE
mydb        1/1     Running   0          15m
wordpress   1/1     Running   0          6s
  0 ✓ 17:46:25 root@k8s-master2-102,172.17.0.1:~ #

```

需要更改wordpress设置重新创建pod，现在终于可以了



5.4 其他debug方式

从你的日志看，WordPress 容器已经重启（或重新部署）并成功运行，但之前的 500 错误依然没有明显的报错日志出现在 stdout 中。在 PHP-Apache 架构中，详细的数据库连接错误通常被记录在容器内的 **错误日志文件** 中，而不是直接显示在 `kubectl logs` 里。

请按照以下三个具体步骤定位最终原因：

1. 查看容器内部的 Apache 错误日志

这是最直接能看到“为什么报错”的地方。执行以下命令：

```
kubectl exec -it wordpress -n wordpress -- tail -f /var/log/apache2/error.log

```

然后刷新你的浏览器访问地址。

- 如果看到 `Access denied for user 'wpuser'@'10.0.0.105'`，说明是**权限问题**。
- 如果看到 `Connection refused` 或 `Connection timed out`，说明是**网络问题**。

2. 测试数据库端口的可达性

由于你使用了 `hostNetwork: true`，网络拓扑变得稍微复杂。请在 WordPress 容器内直接探测数据库端口：

```
# 进入容器
kubectl exec -it wordpress -n wordpress -- bash

# 尝试连接数据库 IP 和端口 (3306)
cat < /dev/tcp/192.168.0.11/3306

```

- **如果没有任何反应（卡住）：** 说明网络不通。请检查你的 Node 节点（k8s-node2 和 k8s-node3）之间的防火墙（iptables/firewalld）是否允许跨主机的容器网段通信。
- **如果提示协议乱码（类似 8.0.29...）：** 说明网络通了，问题在数据库账户。

---

3. 终极修复：MySQL 8.0 的“加密插件”问题

WordPress 默认的 PHP 驱动有时不支持 MySQL 8.0 的 `caching_sha2_password` 认证方式。即便你账号密码对，也会连接失败。

请务必在数据库中执行以下命令（强制降级认证方式）：

```
# 进入数据库
kubectl exec -it mydb -n wordpress -- mysql -uroot -p654321

# 执行以下 SQL 修复
mysql> ALTER USER 'wpuser'@'%' IDENTIFIED WITH mysql_native_password BY '123456';
mysql> FLUSH PRIVILEGES;

```

💡 为什么数据库里 `show tables` 是空的？

这是正常的！

1. WordPress 只有在**第一次成功连接上**数据库后，才会执行安装程序并创建表。
2. 只要你访问时还是 500 错误，说明它**还没连接成功**，所以表是空的。

总结下一步操作：

请先执行 **步骤 1** 获取 `error.log` 中的具体报错信息，把那个报错贴出来，我能立刻告诉你哪里配置错了。如果懒得看日志，请直接执行 **步骤 3** 往往能解决 90% 的 MySQL 8.0 连接问题。

6. 如何设置kubelet bash completion

设置 `kubelet` 的 Bash 自动补全（completion）非常简单，其逻辑与 `kubectl` 的设置基本一致。

以下是为 Bash 环境配置自动补全的步骤：

6.1. 1. 临时生效（当前会话）

如果你只需要在当前的终端窗口中使用，直接运行：

```
source <(kubelet completion bash)

```

6.2. 2. 永久生效（推荐）

为了每次打开终端都能自动加载补全脚本，你需要将其添加到 `.bashrc` 文件中。

6.2.1. 方法 A：直接写入 `.bashrc`

这种方法最直接，不依赖外部文件：

```
echo 'source <(kubelet completion bash)' >> ~/.bashrc
source ~/.bashrc

```

6.2.2. 方法 B：存入系统补全目录（更规范）

如果你有 root 权限，将其放入系统的补全目录可以方便多用户使用或保持配置简洁：

```
kubelet completion bash | sudo tee /etc/bash_completion.d/kubelet > /dev/null
source /etc/bash_completion.d/kubelet

```

6.3. 3. 注意事项与排错

6.3.1. 确保已安装 `bash-completion`

`kubelet completion` 依赖于系统的 `bash-completion` 工具包。如果执行后没有反应，请检查是否已安装该包：

- Debian/Ubuntu: `sudo apt-get install bash-completion`
- CentOS/RHEL/Fedora: `sudo yum install bash-completion`

安装后，确保你的 `~/.bashrc` 中包含以下内容（通常系统会自动添加）：

```
if [ -f /usr/share/bash-completion/bash_completion ]; then
  . /usr/share/bash-completion/bash_completion
fi

```

6.3.2. 验证是否成功

输入 `kubelet` 后按两次 Tab 键。如果你能看到类似下面的选项，说明配置成功了：

```
$ kubelet --[TAB][TAB]
--address                          --experimental-mounter-path
--anonymous-auth                   --experimental-qos-reserved
--authentication-token-webhook     --fail-swap-on
...

```

> **小贴士：** 别忘了顺手把 `kubectl` 的补全也配上，命令是一模一样的：`echo 'source <(kubectl completion bash)' >> ~/.bashrc`。

7. Pod Network Access Troubleshooting

7.1. 问题描述：

在master02节点上访问node3的pod没有返回然和信息，但是在node3上直接访问pod确实可以访问到

```

  0 ✓ 19:33:38 root@k8s-master2-102,172.17.0.1:~ # kubectl get pod pod-with-cmd-and-args  -o wide
NAME                    READY   STATUS    RESTARTS   AGE   IP              NODE            NOMINATED NODE   READINESS GATES
pod-with-cmd-and-args   1/1     Running   0          18s   192.168.0.19    k8s-node3-106   <none>           <none>
  0 ✓ 19:33:44 root@k8s-master2-102,172.17.0.1:~ # kubectl exec pod-with-cmd-and-args  -- ps aux
PID   USER     TIME  COMMAND
    1 root      0:00 python3 /usr/local/bin/demo.py -p 8080
    7 root      0:00 ps aux
  0 ✓ 19:34:07 root@k8s-master2-102,172.17.0.1:~ # kubectl exec pod-with-cmd-and-args -- hostname -i
192.168.0.19
  0 ✓ 19:34:33 root@k8s-master2-102,172.17.0.1:~ # curl 192.168.0.19:8080
  0 ✓ 19:34:47 root@k8s-master2-102,172.17.0.1:~ #  curl 192.168.0.19:8080
  0 ✓ 19:35:44 root@k8s-master2-102,172.17.0.1:~ # ping 192.168.0.19
PING 192.168.0.19 (192.168.0.19) 56(84) bytes of data.
64 bytes from 192.168.0.19: icmp_seq=1 ttl=63 time=0.627 ms
64 bytes from 192.168.0.19: icmp_seq=2 ttl=63 time=0.419 ms
^C
--- 192.168.0.19 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 0.419/0.523/0.627/0.104 ms
  0 ✓ 19:35:53 root@k8s-master2-102,172.17.0.1:~ # cat pod-with-cmd-a
pod-with-cmd-admin-box.yaml  pod-with-cmd-and-args.yaml
  0 ✓ 19:35:53 root@k8s-master2-102,172.17.0.1:~ # cat pod-with-cmd-and-args.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-with-cmd-and-args
spec:
  containers:
  - name: pod-test
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1
    imagePullPolicy: IfNotPresent
    command: ['/bin/sh','-c']
    args: ['python3 /usr/local/bin/demo.py -p 8080']
  0 ✓ 19:36:31 root@k8s-master2-102,172.17.0.1:~ #

  0 ✓ 19:21:08 root@k8s-node3-106,172.17.0.1:~ #  curl 192.168.0.19:8080
kubernetes pod-test v0.1!! ClientIP: 10.0.0.106, ServerName: pod-with-cmd-and-args, ServerIP: 192.168.0.19!
  0 ✓ 19:35:27 root@k8s-node3-106,172.17.0.1:~ #
为啥我在master02上不能访问这个pod呢
```

7.2. 排查过程

如何排查，步骤如下：

这种情况在 Kubernetes 集群中其实非常典型，通常与 **CNI（容器网络接口）的路由实现** 或 **网络插件的具体配置** 有关。

从你的输出看，`ping` 能通但 `curl` 没反应（且没有报错），加上你在 `node3`（Pod 所在的节点）上访问是正常的，这说明 Pod 本身服务没问题，问题出在**跨节点的 Pod 网络通信**上。

以下是导致你在 `master02` 上无法访问该 Pod 的主要原因及排查方向：

7.2.1. 1. CNI 网络插件未完全覆盖所有节点

如果你的集群使用的是 **Flannel (UDP/VXLAN)**、**Calico** 或 **Cilium**，网络插件负责在所有节点之间建立“大二层”网络。

- **现象**：`master02` 可能没有正确加入容器网络的路由表。
- **排查**：在 `master02` 上运行 `ip route`。
- 看是否存在去往 `192.168.0.0/24`（Pod 网段）的路由条目。
- 如果路由条目的网关（Gateway）指向错误，或者压根没有相关路由，流量就发不出去。

7.2.2. 2. Master 节点未运行网络插件 Pod

在某些生产环境中，为了安全或性能，Master 节点可能设置了 `NoSchedule` 污点，导致 CNI 插件（如 Flannel 的 DaemonSet）没有在 Master 节点上运行。

- **排查**：`kubectl get pods -n kube-system -o wide`
- **检查**：确保你的网络插件（如 `kube-flannel-ds` 或 `calico-node`）在 `k8s-master2-102` 上处于 **Running** 状态。如果 Master 上没运行网络插件，它就不知道如何封包转发到 Pod 所在的 Node。

7.2.3. 3. 安全组或防火墙（iptables/nftables）限制

虽然 `ping` 使用的是 ICMP 协议，而 `curl` 使用的是 TCP 协议。

- **现象**：有时候防火墙策略允许 ICMP 探测，但拦截了特定端口（8080）或协议（TCP）。
- **检查**：
- 检查 Master 和 Node 节点上的 `ufw` 或 `firewalld` 是否关闭。
- 检查云服务器（阿里云）的内部安全组规则，是否允许 Master 节点与 Node 节点之间全协议通信。

7.2.4. 4. 路由不对称或反向路径过滤（rp_filter）

如果 `master02` 有多个网卡，流量可能从 A 网卡发出，但响应从 B 网卡回来，导致 Linux 内核认为这是非法包而丢弃。

- **检查**：在 Master 上执行 `sysctl -a | grep "\.rp_filter"`。
- **解决**：尝试临时将相应网卡的 `rp_filter` 设置为 0 或 2（松散模式）。

7.2.5. 5. 代理/环境变量干扰

注意到你的 `curl` 没有立即返回 `Connection refused` 而是卡住或无输出。

- **排查**：在 `master02` 执行 `env | grep -i proxy`。
- **风险**：如果设置了全局 `http_proxy`，`curl` 可能会尝试走代理服务器而不是直接访问 Pod IP。

7.2.6. 建议操作步骤：

1. **对比路由**：在 `master02` 执行 `ip route`，确认 `192.168.0.0/24` 的下一跳（via）是否指向 `k8s-node3-106` 的物理 IP。
2. **抓包分析**：在 `node3` 上抓包看流量是否到达：

```
tcpdump -i any host 192.168.0.19 and port 8080 -n

```

然后在 `master02` 发起 `curl` 。如果 `node3` 没收到包，说明是发送端（Master）**路由问题；如果收到了包但没回包，说明是**接收端（Node/Pod）回程路由或防火墙问题。

补充提问：你在 `master02` 上执行 `curl -v 192.168.0.19:8080` 具体的输出是什么？是卡住不动（Timeout）还是立即报错？

```
1
```

7.3. 实际排查过程

发现是用了代理，接没有报错，也没有任何消息，所以需要把192.168.0.0/24这个网络段加入到no_proxy的列表中

```

  0 ✓ 19:36:31 root@k8s-master2-102,172.17.0.1:~ # curl -v 192.168.0.19:8080
* Uses proxy env variable no_proxy == 'localhost,127.0.0.1,10.244.0.0/16,10.96.0.0/12,172.17.0.0/16,10.0.0.101,svc,cluster.local,kubeapi.zhong.org'
* Uses proxy env variable http_proxy == 'http://192.168.71.108:8899'
*   Trying 192.168.71.108:8899...
* Connected to 192.168.71.108 (192.168.71.108) port 8899
> GET http://192.168.0.19:8080/ HTTP/1.1
> Host: 192.168.0.19:8080
> User-Agent: curl/8.5.0
> Accept: */*
> Proxy-Connection: Keep-Alive
>
< HTTP/1.1 502 Bad Gateway
< Connection: keep-alive
< Keep-Alive: timeout=4
< Proxy-Connection: keep-alive
< Content-Length: 0
<
* Connection #0 to host 192.168.71.108 left intact
  0 ✓ 19:38:18 root@k8s-master2-102,172.17.0.1:~ # env | grep proxy
no_proxy=localhost,127.0.0.1,10.244.0.0/16,10.96.0.0/12,172.17.0.0/16,10.0.0.101,svc,cluster.local,kubeapi.zhong.org
https_proxy=http://192.168.71.108:8899
http_proxy=http://192.168.71.108:8899

  1 ✗ 19:39:52 root@k8s-master2-102,172.17.0.1:~ # cd /data/scripts/
  0 ✓ 19:39:57 root@k8s-master2-102,172.17.0.1:/data/scripts # ll
总计 12
drwxr-xr-x 2 root root 4096  5月 14 14:23 ./
drwxr-xr-x 3 root root 4096  4月 24 13:54 ../
-rwxr-xr-x 1 root root 2303  5月 14 14:23 proxy_mgr.sh*
  0 ✓ 19:39:57 root@k8s-master2-102,172.17.0.1:/data/scripts # vim proxy_mgr.sh
  0 ✓ 19:40:37 root@k8s-master2-102,172.17.0.1:/data/scripts # ./proxy_mgr.sh start
>>> 正在启用代理设置...
[OK] 已配置 docker
[OK] 已配置 containerd
[OK] 已配置 kubelet
正在重载 systemd 并重启服务...
>>> 代理已开启。请执行 'source /etc/profile.d/k8s_proxy.sh' 更新当前 Shell 会话。
  0 ✓ 19:40:57 root@k8s-master2-102,172.17.0.1:/data/scripts # source /etc/profile
profile    profile.d/
```

```
  0 ✓ 19:40:57 root@k8s-master2-102,172.17.0.1:/data/scripts # source /etc/profile
profile    profile.d/
  0 ✓ 19:40:57 root@k8s-master2-102,172.17.0.1:/data/scripts # source /etc/profile.d/k8s_proxy.sh
  0 ✓ 19:41:12 root@k8s-master2-102,172.17.0.1:/data/scripts # env | grep proxy
no_proxy=localhost,127.0.0.1,10.244.0.0/16,192.168.0.0/24,10.96.0.0/12,172.17.0.0/16,10.0.0.101,svc,cluster.local,kubeapi.zhong.org
https_proxy=http://192.168.71.108:8899
http_proxy=http://192.168.71.108:8899
  0 ✓ 19:41:18 root@k8s-master2-102,172.17.0.1:/data/scripts # curl -v 192.168.0.19:8080
* Uses proxy env variable no_proxy == 'localhost,127.0.0.1,10.244.0.0/16,192.168.0.0/24,10.96.0.0/12,172.17.0.0/16,10.0.0.101,svc,cluster.local,kubeapi.zhong.org'
*   Trying 192.168.0.19:8080...
* Connected to 192.168.0.19 (192.168.0.19) port 8080
> GET / HTTP/1.1
> Host: 192.168.0.19:8080
> User-Agent: curl/8.5.0
> Accept: */*
>
* HTTP 1.0, assume close after body
< HTTP/1.0 200 OK
< Content-Type: text/html; charset=utf-8
< Content-Length: 109
< Server: Werkzeug/1.0.0 Python/3.8.2
< Date: Thu, 14 May 2026 11:41:31 GMT
<
kubernetes pod-test v0.1!! ClientIP: 192.168.0.3, ServerName: pod-with-cmd-and-args, ServerIP: 192.168.0.19!
* Closing connection
  0 ✓ 19:41:32 root@k8s-master2-102,172.17.0.1:/data/scripts # curl 192.168.0.19:8080
kubernetes pod-test v0.1!! ClientIP: 192.168.0.3, ServerName: pod-with-cmd-and-args, ServerIP: 192.168.0.19!
  0 ✓ 19:41:44 root@k8s-master2-102,172.17.0.1:/data/scripts #
```

8. pod name需要小写

```
  0 ✓ 21:12:15 root@k8s-master2-102,172.17.0.1:~ # kubectl apply -f pod-hostport.yaml
The Pod "pod-hostPort-demo" is invalid: metadata.name: Invalid value: "pod-hostPort-demo": a lowercase RFC 1123 subdomain must consist of lower case alphanumeric characters, '-' or '.', and must start and end with an alphanumeric character (e.g. 'example.com', regex used for validation is '[a-z0-9]([-a-z0-9]*[a-z0-9])?(\.[a-z0-9]([-a-z0-9]*[a-z0-9])?)*')
  1 ✗ 21:12:19 root@k8s-master2-102,172.17.0.1:~ # cat pod-hostport.yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-hostPort-demo
spec:
  containers:
  - name: demo-env
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1
    env:
    - name: PORT
      value: "9999"
    ports:
    - name: http
      containerPort: 9999
      hostPort: 8888
  0 ✓ 21:12:33 root@k8s-master2-102,172.17.0.1:~ #

```

9. hostPort使用宿主机的端口

如下pod的yaml文件

```yaml
kind: Pod
metadata:
  name: pod-hostport-demo
spec:
  containers:
  - name: demo-env
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/pod-test:v0.1
    env:
    - name: PORT
      value: "9999"
    ports:
    - name: http
      containerPort: 9999
      hostPort: 8888

```

本质就是利用iptables加了一条DNAT的转化规则

```
#在宿主机上只能访问外部ip地址
C:\Users\clock>curl 10.0.0.105:8888
kubernetes pod-test v0.1!! ClientIP: 10.0.0.1, ServerName: pod-hostport-demo, ServerIP: 192.168.0.21!

C:\Users\clock>curl 192.168.0.21:9999
^C
C:\Users\clock>

#在master2上访问内部和外部地址都可以
  7 ✗ 21:54:33 root@k8s-master2-102,172.17.0.1:~ # curl 10.0.0.105:8888
kubernetes pod-test v0.1!! ClientIP: 10.0.0.102, ServerName: pod-hostport-demo, ServerIP: 192.168.0.21!
  0 ✓ 21:54:42 root@k8s-master2-102,172.17.0.1:~ # curl 192.168.0.21:9999
kubernetes pod-test v0.1!! ClientIP: 192.168.0.3, ServerName: pod-hostport-demo, ServerIP: 192.168.0.21!
  0 ✓ 21:54:53 root@k8s-master2-102,172.17.0.1:~ #

#在node2上查看iptables规则
  1 ✗ 21:43:39 root@k8s-node2-105,172.17.0.1:~ # iptables -vnL -t nat| grep DNAT|grep 8888
    9   500 DNAT       6    --  *      *       0.0.0.0/0            0.0.0.0/0            tcp dpt:8888 to:192.168.0.21:9999
  0 ✓ 21:49:36 root@k8s-node2-105,172.17.0.1:~ #

```

10. docker服务停止引起的k8s集群故障

为啥好好的docker服务会停止，而且一旦一个master的服务听了，其他的也不行了，反之，如果重启了，其他的好像又都好了，该如何避免和排查这些故障

这是一个非常深刻的问题。在 Kubernetes 架构中，Master 节点是集群的大脑，而 Docker（或 Containerd）则是支撑大脑运行的“心脏”。

以下是针对你的疑问进行的深度拆解，以及如何构建一个稳健集群的建议。

10.1. 为什么 Docker 会好端端地停止?

Docker 作为一个守护进程，通常不会无故自杀。常见的诱因包括：

- **资源耗尽 (OOM Kill)**：如果 Master 节点的内存被某个进程挤占，系统内核为了保命，可能会杀掉内存占用较高的 Docker 相关进程。
- **磁盘空间满**：这是最常见的原因。`/var/lib/docker` 所在的磁盘如果被日志、镜像或临时文件填满，Docker 往往会直接陷入僵死或崩溃状态。
- **systemd 配置冲突**：正如你之前提到的代理设置。如果你在系统层面改了环境变量，Docker 在重启时可能因为无法绕过代理去连接本地的 `unix socket` 或内部 registry 而启动失败。
- **驱动不一致**：Cgroup Driver（cgroupfs vs systemd）不一致会导致 Kubelet 与 Docker 持续“打架”，增加系统负担。

10.2. 为什么一个 Master 挂了，感觉整个集群都“瘫”了?

这涉及到 Kubernetes **控制平面的级联反应**。

- **API Server 的中心地位**：API Server 是唯一与 `etcd` 数据库通信的组件。你的 `kubectl`、Kubelet、甚至节点间的网络插件（Calico/Flannel）都要找 API Server。如果这个 Master 上的 Docker 挂了，API Server 容器就没了。
- **静态 Pod (Static Pods) 的特殊性**：Master 核心组件（API, Scheduler, Controller-Manager）通常作为静态 Pod 运行，由 Kubelet 直接管理。Kubelet 发现 Docker 挂了，就无法拉起这些核心组件。
- **连锁崩溃**：
- **没有 API**：节点无法上报心跳。

- **没有 Controller-Manager**：无法处理节点失联后的副本迁移。
- **没有 Scheduler**：新的 Pod 无法调度。
- 虽然 Worker 上的业务容器可能还在跑，但你已经失去了对集群的“控制权”。

10.3. 如何避免这些故障？（高可用设计）

要让“重启一个，其他的都好了”这种不稳定感消失，你需要做 **HA（高可用）**。

10.3.1. A. 真正的多 Master 高可用

你现在可能有多个 Master，但如果它们之间没有负载均衡，你还是在连接某个固定的 IP。

- **使用 VIP (Virtual IP)**：通过 `Keepalived` + `HAProxy` 给多个 Master 的 API Server 提供一个统一的访问入口。
- **避免单点**：当 master1 挂了，VIP 自动漂移到 master2，`kubectl` 甚至感觉不到断开。

10.3.2. B. 资源预留 (Kube-reserved)

配置 Kubelet 预留一部分系统资源给底层服务：

```
  --kube-reserved=cpu=500m,memory=1gi,ephemeral-storage=1gi
  --system-reserved=cpu=500m,memory=1gi,ephemeral-storage=1gi

```

这能防止业务 Pod 把 Master 节点的内存撑爆。

10.3.3. C. 监控与告警

- **磁盘预警**：重点监控 `/var/lib/docker` 和 `/var/log`。
- **进程监控**：监控 `dockerd`、`cri-dockerd`、`kubelet` 的存活状态。

10.4. 故障排查标准流程 (SOP)

当 `kubectl` 报错 `EOF` 或无法连接时，按此顺序排查：

1. **查系统层**：

- `df -h` （看磁盘满没满）
- `free -m` （看内存够不够）
- `uptime` （看负载是否过高）

2. **查服务层**：

- `systemctl status docker`
- `systemctl status cri-dockerd`
- `systemctl status kubelet`

3. **查核心容器**：

- 如果 Docker 还在跑，看 API Server 容器： `docker ps | grep apiserver`

4. **看系统日志**：

- `journalctl -u docker -f`
- `journalctl -u kubelet -f`
- `/var/log/syslog` 或 `/var/log/messages`

一句话建议： 检查一下你 Master 节点的磁盘占用情况（ `df -h` ），我怀疑你的日志或 Docker 镜像占满了根分区。

你目前这套集群是个人练习用的单点测试环境，还是正式的生产/开发环境?

11. kubectl exec vs kubectl debug

```yaml
  0 ✓ 22:20:50 root@k8s-master1-101,172.17.0.1:~ # kubectl exec -it ephemeral-demo -- /bin/bash
OCI runtime exec failed: exec failed: unable to start container process: exec: "/bin/bash": stat /bin/bash: no such file or directory
command terminated with exit code 127
127 ✗ 22:21:08 root@k8s-master1-101,172.17.0.1:~ # kubectl exec -it ephemeral-demo -- sh
OCI runtime exec failed: exec failed: unable to start container process: exec: "sh": executable file not found in $PATH
command terminated with exit code 127
127 ✗ 22:21:18 root@k8s-master1-101,172.17.0.1:~ #

```

简单来说，`kubectl exec` 是在**现有容器内部**运行命令，而 `kubectl debug` 则是通过**额外增加一个临时容器**来观察或修复现有的 Pod。

以下是它们的深度对比和具体用法：

11.1. 核心区别：工作原理不同

特性	`kubectl exec`	`kubectl debug`（临时容器模式）
原理	在 Pod 中**已有的容器**中启动一个新进程。	在 Pod 中**新建一个容器**（Ephemeral Container）。
依赖性	依赖镜像内的二进制文件（如 `sh`，`ls`）。	不依赖原镜像，可以使用 `busybox` 等工具镜像。
副作用	几乎没有，只是多了一个进程。	会修改 Pod 的 Spec（增加了一个临时容器）。
主要场景	日常查看日志、进入应用执行简单指令。	诊断“瘦镜像”（如 Distroless）或已崩溃的容器。

11.2. 为什么你刚才的命令失败了?

当你执行 `kubectl exec ... -- sh` 时，Kubernetes 会在你的业务容器里寻找 `/bin/sh` 。

- **现代安全实践**：为了减小攻击面和镜像体积，很多生产镜像（如 Go 编译出的镜像或 Distroless）删除了所有不必要的工具，包括 Shell。
- **结果**：容器里根本没有 `sh` 这个可执行文件，所以报错 `executable file not found` 。

11.3. `kubectl debug` 的精妙之处

在你使用的命令中：
`kubectl debug -it ephemeral-demo --image=busybox:1.32.0 --target=ephemeral-demo`

- `--image=busybox` ： 既然原容器没工具，我就带个“工具箱”进去。
- `--target` ： 这是关键。它利用了 Linux 的 **Namespace 共享**。虽然 debug 容器和业务容器是两个独立的容器，但通过 `--target` ，debug 容器可以看到业务容器的进程（PID Namespace），甚至通过 `/proc/1/root` 访问业务容器的文件系统。

```yaml
127 ✗ 22:21:18 root@k8s-master1-101,172.17.0.1:~ # kubectl debug -it ephemeral-demo --image=busybox:1.32.0 --target=ephemeral-demo
Targeting container "ephemeral-demo". If you don't see processes from this container it may be because the container runtime doesn't support this feature.
--profile=legacy is deprecated and will be removed in the future. It is recommended to explicitly specify a profile, for example "--profile=general".
Defaulting debug container name to debugger-vgh7c.
All commands and output from this session will be recorded in container logs, including credentials and sensitive information passed through the command prompt.
If you don't see a command prompt, try pressing enter.
/ # ps
PID   USER     TIME  COMMAND
    1 65535     0:00 /pause
   19 root      0:00 sh
   25 root      0:00 ps
/ # ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: tunl0@NONE: <NOARP> mtu 1480 qdisc noop qlen 1000
    link/ipip 0.0.0.0 brd 0.0.0.0
3: eth0@if18: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 1480 qdisc noqueue qlen 1000
    link/ether 26:d2:1c:5b:a5:ee brd ff:ff:ff:ff:ff:ff
    inet 192.168.0.38/32 scope global eth0
       valid_lft forever preferred_lft forever
    inet6 fe80::24d2:1cff:fe5b:a5ee/64 scope link
       valid_lft forever preferred_lft forever
/ #
```

11.4. 常用用法总结

11.4.1. `kubectl exec`：常规操作

如果你确定容器里有 shell（如 `ubuntu` 或 `alpine` 镜像）：

```
# 进入交互式 shell
kubectl exec -it <pod-name> -- /bin/bash

# 不进入容器，直接执行查看命令
kubectl exec <pod-name> -- ls /app/config

```

11.4.2. `kubectl debug`：高级诊断

除了你刚才用的“添加临时容器”法，它还有两个非常强大的变体：

A. 复制 Pod 并修改镜像（修复启动失败的 Pod）

如果一个 Pod 启动就崩溃（CrashLoopBackOff），你无法 `exec` 进去。这时可以复制它并换个启动命令：

```
kubectl debug <pod-name> -it --copy-to=debug-pod --container=main --sh

```

B. 诊断节点（直接登录 Node 节点）

如果不是 Pod 出问题，而是服务器（Node）挂了，你可以创建一个拥有宿主机权限的 Pod：

```
kubectl debug node/<node-name> -it --image=ubuntu
# 进入后执行 chroot /host 即可像 SSH 一样操作物理机

```

11.4.3. 范例

debug nginx示例

```yaml
  0 ✓ 22:31:54 root@k8s-master1-101,172.17.0.1:~ # kubectl run mynginx --image=registry.cn-beijing.aliyuncs.com/wangxiaochun/nginx:1.22.0
pod/mynginx created
  0 ✓ 22:37:20 root@k8s-master1-101,172.17.0.1:~ # kubectl exec -it mynginx -- sh
# ip a
sh: 1: ip: not found
# ps
sh: 2: ps: not found
# exit
command terminated with exit code 127
127 ✗ 22:37:55 root@k8s-master1-101,172.17.0.1:~ # kubectl debug -it mynginx --image=registry.cn-beijing.aliyuncs.com/wangxiaochun/busybox:1.32.0 --target=mynginx
Targeting container "mynginx". If you don't see processes from this container it may be because the container runtime doesn't support this feature.
--profile=legacy is deprecated and will be removed in the future. It is recommended to explicitly specify a profile, for example "--profile=general".
Defaulting debug container name to debugger-w25kc.
All commands and output from this session will be recorded in container logs, including credentials and sensitive information passed through the command prompt.
If you don't see a command prompt, try pressing enter.
/ # ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: tunl0@NONE: <NOARP> mtu 1480 qdisc noop qlen 1000
    link/ipip 0.0.0.0 brd 0.0.0.0
3: eth0@if10: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 1480 qdisc noqueue qlen 1000
    link/ether 36:29:14:ab:9c:04 brd ff:ff:ff:ff:ff:ff
    inet 192.168.0.39/32 scope global eth0
       valid_lft forever preferred_lft forever
    inet6 fe80::3429:14ff:feab:9c04/64 scope link
       valid_lft forever preferred_lft forever
/ # ps
PID   USER     TIME  COMMAND
    1 root      0:00 nginx: master process nginx -g daemon off;
   32 101       0:00 nginx: worker process
   33 101       0:00 nginx: worker process
```

```
    40 root      0:00 sh
    46 root      0:00 ps
/ #
0 ✓ 22:40:09 root@k8s-master1-101,172.17.0.1:~ # kubectl get pod mynginx -o yaml |grep -A10 ephemeralContainers
  ephemeralContainers:
  - image: registry.cn-beijing.aliyuncs.com/wangxiaochun/busybox:1.32.0
    imagePullPolicy: IfNotPresent
    name: debugger-w25kc
    resources: {}
    stdin: true
    targetContainerName: mynginx
    terminationMessagePath: /dev/termination-log
    terminationMessagePolicy: File
    tty: true
  nodeName: k8s-node2-105
  0 ✓ 22:40:11 root@k8s-master1-101,172.17.0.1:~ #
```

11.4.4. 总结建议

- **优先使用 `exec**`：因为它快、轻量，不改变 Pod 状态。
- `debug` **是救命稻草**：当你发现镜像里啥都没有（精简镜像），或者 Pod 已经挂了无法进入时，再祭出 `kubectl debug`。

你现在的实验环境里，`kubectl debug` 配合 `--target` 是观察业务容器运行状态（如查看内存占用、网络连接）最优雅的方式。